

AFLOW: 自动化智能体 workflow 生成

Jiayi Zhang^{1,2*}, Jinyu Xiang^{1*}, Zhaoyang Yu³, Fengwei Teng³, Xiong-Hui Chen⁴,
Jiaqi Chen⁵, Mingchen Zhuge⁶, Xin Cheng³, Sirui Hong¹, Jinlin Wang¹, Bingnan Zheng⁵,
Bang Liu⁷, Yuyu Luo^{2,8†}, Chenglin Wu^{1†}

¹DeepWisdom, ²The Hong Kong University of Science and Technology (Guangzhou),

³Renmin University of China, ⁴Nanjing University, ⁵Fudan University,

⁶King Abdullah University of Science and Technology, ⁷Université de Montréal & Mila,

⁸The Hong Kong University of Science and Technology

摘要

大语言模型 (Large Language Models) 在解决跨领域复杂任务方面展现出显著潜力, 通常通过遵循详细指令和操作序列的智能体 workflow 来实现。然而, 构建这些 workflow 需要大量人工努力, 限制了可扩展性和泛化能力。近期研究试图自动化生成和优化这些 workflow, 但现有方法仍依赖初始手动设置, 未能实现完全自动化且有效的 workflow 生成。为应对这一挑战, 我们将 workflow 优化重新表述为对代码表示的 workflow 进行搜索的问题, 其中调用大语言模型的节点通过边连接。我们提出 **AFLOW**, 一个自动化框架, 利用蒙特卡洛树搜索高效探索该空间, 通过代码修改、树结构经验和执行反馈迭代优化 workflow。在六个基准数据集上的实证评估证明了 **AFLOW** 的有效性, 相比最先进的基线平均提升 5.7%。此外, **AFLOW** 使较小模型在特定任务上以 GPT-4o 推理成本的 4.55% (以美元计) 超越 GPT-4o。代码可在 <https://github.com/FoundationAgents/AFlow> 获取。

1 引言

大语言模型 (Large Language Models) 已成为解决跨领域复杂任务的强大工具, 涵盖代码生成、数据分析、决策制定和问答等 (Liu et al., 2024; Li et al., 2024a; Zhu et al., 2024; Xie et al., 2024b; Sun et al., 2024; Wang et al., 2024b; Song et al., 2023; Xie et al., 2024a; Zhong et al., 2024a)。然而, 大语言模型的快速发展在很大程度上依赖于人工设计的智能体 workflow——即伴随详细指令的结构化大语言模型调用序列。设计和优化这些 workflow 需要大量人工努力, 这限制了大语言模型对新复杂领域的可扩展性和适应性, 并阻碍了其在多样化任务间迁移技能的能力 (Tang et al., 2024)。

近期研究致力于自动发现有效的智能体 workflow, 以减少对人工干预的依赖 (Khatab 等, 2024; Y`uksekg`on`ul 等, 2024; Liu 等, 2023; Hu 等, 2024)。尽管取得了这些进展, 完全自动化尚未实现。例如, Khatab 等 (2024) 在自动提示优化之前需要手动设置 workflow。类似地, Y`uksekg`on`ul 等 (2024) 和 Zhuge 等 (2024) 提出的方法未能捕捉到广泛任务所需的工作流多样性 (Yu 等, 2023; Yang 等, 2024b; Sun 等, 2023), 因为它们的优化目标难以表示可能工作流的广度。在这些自动化系统中无法有效建模多样化的 workflow 结构, 限制了它们的实用性和影响力。ADAS (Hu 等, 2024) 使用代码表示 workflow, 实现了

* 这些作者对本文贡献相同。† 通讯作者: Yuyu Luo (电子邮箱: yuyuluo@hkust-gz.edu.cn), Chenglin Wu (电子邮箱: alexanderwu@deepwisdom.ai)

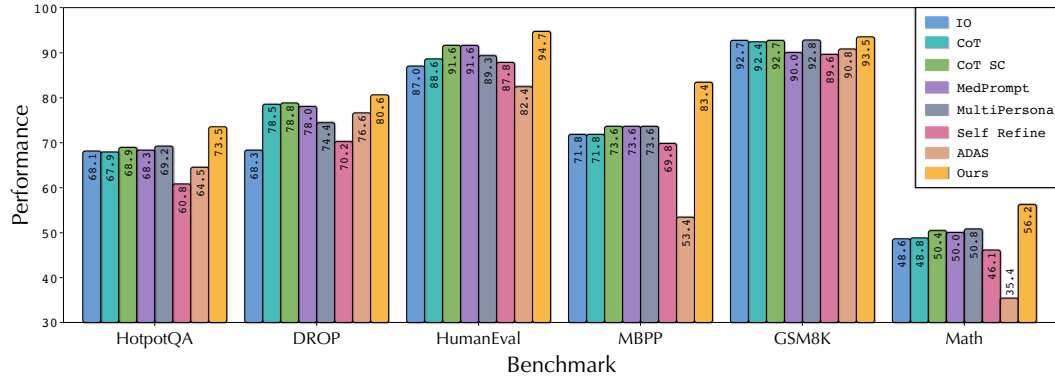


图 1：与其他方法的性能比较。为评估方法性能，我们在不同数据集上采用多种度量：Math 和 GSM8K 使用解题率，HotpotQA 和 DROP 使用 F1 分数，HumanEval 和 MBPP 使用 pass@1。我们的 AFLOW（以黄色高亮）在所有六个基准上始终优于所有自动化工作流优化方法和人工设计方法。

相对完整的表示。然而，由于其线性启发式搜索算法的效率限制，ADAS 难以在有限的迭代次数内生成有效的工作流。这凸显了对更有效技术来表示和自动化生成智能体工作流的需求，这将加速大语言模型在各领域的应用。

针对这些挑战，我们提出一个创新的框架，用于自动生成智能体工作流。我们的核心思想是将工作流建模为一系列相互连接的调用大语言模型的节点，其中每个节点代表一个大语言模型动作，边定义这些动作之间的逻辑、依赖关系和流程。这种结构将工作流转化为一个巨大的搜索空间，涵盖各种潜在的配置。我们的目标是高效地导航该空间，自动生成优化的工作流，以最大化任务性能，同时最小化人工干预。

然而，任务的多样性与复杂性带来了显著挑战。具体而言，每个任务可能具有不同的需求、操作和依赖关系，这使得以统一且灵活的方式表示它们变得困难（Chen 等，2021；Cobbe 等，2021；Yang 等，2018；Luo 等，2018）。此外，可能的工作流搜索空间包含海量的代码结构和节点配置，几乎是无边无际的，这为高效探索和优化带来了额外挑战。

为应对这些挑战，本文提出 AFLOW，一个基于蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）的框架，旨在系统性地探索和发现最优智能体工作流。AFLOW 将工作流表示为通过基于代码的边连接的灵活节点，这些边封装了逻辑流、条件和依赖关系等可能的关系。这些边使得工作流能够被建模为图（Zhuge 等，2024）或网络（Liu 等，2023），为捕捉大语言模型调用之间的复杂交互提供了强大的结构。

为增强搜索过程并提高效率，AFLOW 引入了新颖的算子概念——预定义、可复用的节点组合，代表常见的智能体操作（例如集成、审查与修订）。这些算子作为构建工作流的基础模块，并被整合到搜索空间中，确保探索过程能够利用已知的有效智能体操作模式。

AFLOW 采用蒙特卡洛树搜索算法来导航这个无限的搜索空间。该框架的工作流优化过程包含多项关键创新：用于节点探索的软混合概率选择机制、由大语言模型驱动节点扩展以引入新的可能性、执行评估以衡量工作流性能，以及经验反向传播以优化未来的搜索迭代。这些技术的结合确保了 AFLOW 能够高效地发现适应多样化任务复杂性的工作流，同时减少对人工干预的依赖。

本文做出以下关键贡献：（1）问题形式化：我们将工作流优化问题形式化，将先前的方法推广为特例。这为节点和工作流优化层面的未来研究提供了统一框架。（2）AFLOW：

我们提出 AFLOW，一种基于蒙特卡洛树搜索的方法，能够在多个领域自动发现有效的工作流，且仅需极少的人工干预。（3）广泛评估：我们在六个基准数据集上评估 AFLOW：HumanEval、MBPP、MATH、GSM8K、HotPotQA 和 DROP。AFLOW 比人工设计的方法高出 5.7%，比现有自动化方法高出 19.5%。值得注意的是，AFLOW 生成的工作流使较小的 LLM 能够超越较大的模型，提供更好的成本性能效率，对实际应用具有重大意义。

2 相关工作

智能体工作流 智能体工作流和自主智能体（Zhuge 等，2023；Hong 等，2024a；Zhang 等，2024c；Wang 等，2023）代表了 LLM 应用的两种不同范式。前者通过预定义流程和多次 LLM 调用静态地完成任务，而后者通过灵活的自主决策动态地解决问题。与需要为环境设计特定动作和决策模式的自主智能体相比，智能体工作流可以基于现有的人类领域经验和迭代优化来构建，具有更高的自动化构建潜力。

智能体工作流大致可分为通用型和领域特定型。通用工作流强调通用的解决问题方法，例如（Wei 等，2022；Wang 等，2022；Madaan 等，2023；Wang 等，2024a）。领域特定工作流专注于构建有效流程以解决特定领域的问题，例如代码生成（Hong 等，2024b；Ridnik 等，2024；Zhong 等，2024a）、数据分析（Xie 等，2024b；Ye 等，2024；Li 等，2024a；Zhou 等，2023）、数学（Zhong 等，2024b；Xu 等，2024）、问答（Nori 等，2023；Zhou 等，2024a）。现有工作已经手动发现了许多有效的智能体工作流，但很难穷尽不同领域的各种任务，这进一步凸显了自动化工作流生成和优化的重要性。

自动化智能体优化 近期工作旨在自动化智能体工作流的设计，可分为三类：自动化提示优化、超参数优化和自动化工作流优化。提示优化（费尔南多等人，2024；尤克塞尔格尼尔等人，2024；杨等人，2024a；哈塔布等人，2024）利用大语言模型在固定工作流内优化提示。超参数优化（萨阿德-法尔孔等人，2024）专注于优化预定义参数。尽管这些方法提升了性能，但在新任务上的泛化能力有限，且通常需要适度的人工努力进行任务特定设计。

自动化工作流优化（李等人，2024b；周等人，2024b；诸葛等人，2024；胡等人，2024）旨在优化整个工作流结构，为全自动生成提供了更大的潜力。近期工作探索了多样的表示和方法。GPTSwarm（诸葛等人，2024）使用图结构与强化学习，但由于图结构的限制，难以表示具有条件状态的工作流。ADAS（胡等人，2024）利用代码结构表示工作流，并将历史工作流存储在表结构中，与我们的目标高度一致。然而，其搜索算法的效率受到限制，因为它在搜索过程中对经验的表示过于简单，导致难以发现有效的工作流。

AFLOW 同样使用代码表示工作流，但更进一步，提供了一种更基础的结构，称为命名节点。该结构包含各种大语言模型调用参数，允许更详细的工作流表示。我们还引入了实现预定义节点组合函数的算子。同时，AFLOW 采用专门设计的蒙特卡洛树搜索算法进行自动化工作流优化，利用树结构经验和执行反馈高效地发现有效工作流。

3 预备知识

本节首先在第 3.1 节中形式化自动化智能体工作流生成问题，然后在第 3.2 节中讨论 AFLOW 的设计考虑。对于本节的核心概念，我们在图 2 中提供了示例说明。

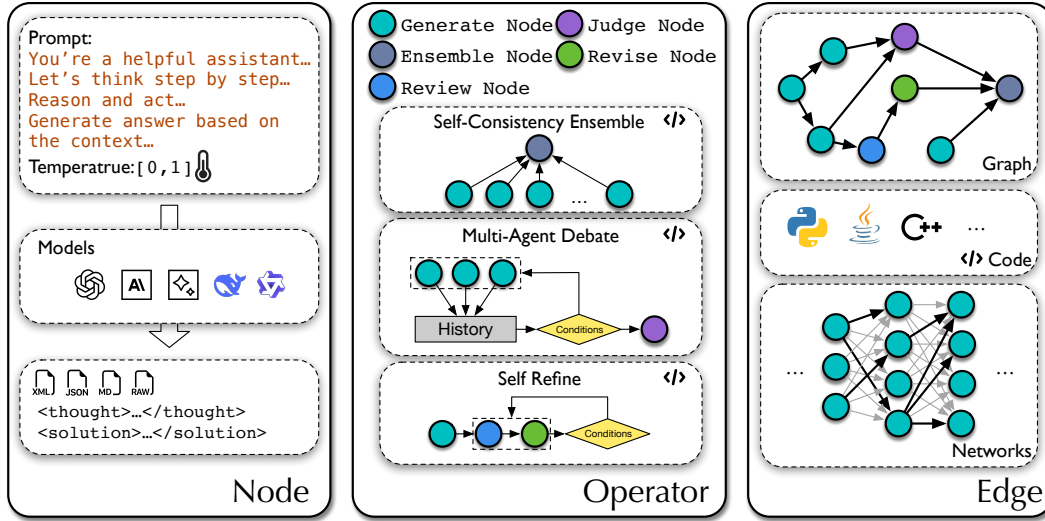


图 2：节点、算子和边的示例。我们展示了节点的可选参数、部分算子的结构以及边的常见表示。

3.1 问题形式化

智能体工作流 我们将智能体工作流（Agentic Workflow） W 定义为一组由边连接、用于定义执行顺序的调用大语言模型（LLM）的节点序列，记作 $\mathcal{N} = \{N_1, N_2, \dots, N_i \dots\}$ 。每个节点 N_i 代表由大语言模型执行的特定操作，并由以下参数表征。节点的代码抽象见附录 A.2。

- 模型 M ：在节点 N_i 处调用的特定语言模型。
- 提示 P ：在每个节点提供给模型的输入或任务描述。
- 温度 τ ：控制节点 N_i 处大语言模型输出随机性的参数。
- 输出格式 F ：模型输出所采用的结构化格式（例如，xml、json、markdown、raw）。受 Tam 等人（2024）的启发，工作流中的节点应提供不同的输出格式。

边 E 表示定义节点关系、控制执行顺序的抽象结构。边 E 可以通过多种结构来表示，例如：

- **图 (Graph) Zhuge 等人 (2024)**：一种灵活的结构，表示节点之间的层次、顺序或并行关系，允许复杂的分支工作流。
- **神经网络 (Neural Network) (Liu 等人, 2023)**：一种能够表示节点之间复杂非线性关系的结构，允许基于输入和反馈的自适应和可学习工作流。
- **代码 (Code) (Hu 等人, 2024)**：一种全面的表示方法，能够表达线性序列、条件逻辑、循环，并整合图或网络结构，为大型语言模型的工作流执行提供最精确的控制。

虽然图结构可以表示工作流关系，但它们需要超越基本有向无环图的复杂扩展（例如，佩特里网、业务流程建模与标注）才能自然地表达并行执行和条件逻辑。神经网络能够实现自适应转换，但缺乏对工作流执行的精确控制。相比之下，代码表示通过标准编程结构天然支持所有这些关系。因此，我们采用代码作为主要边结构，以最大化表达能力。

自动化工作流优化 给定任务 T 和评估函数 G ，工作流优化的目标是发现一个工作流 W ，使得 $G(W, T)$ 最大化。这可以形式化为一个搜索过程，其中算法 A 探索搜索空间 $\mathcal{S}$ 以确定最优工作流配置。工作流优化问题的搜索空间 $\mathcal{S}$ 包含节点参数和边结构的所有可能配置：

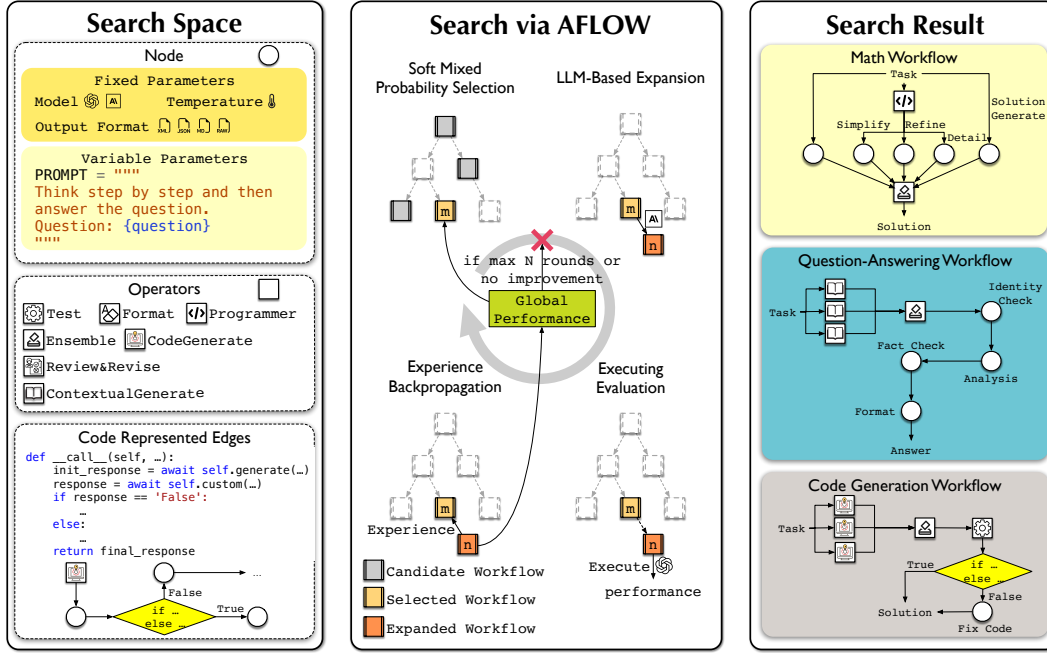


图 3: AFLOW 总体框架：通过设置由仅提示参数灵活的节点、给定的算子集以及表示边的代码组成的搜索空间，AFLOW 在该空间内执行基于蒙特卡洛树搜索的搜索。通过专为工作流优化设计的蒙特卡洛树搜索变体，AFLOW 迭代执行软混合概率选择、基于大型语言模型的扩展、执行评估和经验反向传播的循环，直到达到最大迭代次数或满足收敛条件。

$$\mathcal{S} = \{(\mathcal{N}, E) \mid E \in \mathcal{E}\},$$

where $\mathcal{N} = \{N(M, \tau, P, F) \mid M \in \mathcal{M}, \tau \in [0, 1], P \in \mathcal{P}, F \in \mathcal{F}\}$, with $\mathcal{M}, \mathcal{P}, \mathcal{F}, \mathcal{E}$ representing 分别是可能的语言模型、提示、输出格式和边配置的集合。

通过这种形式化，工作流优化问题可以表示为：

$$\begin{aligned} W &= A(\mathcal{S}, G, T), \\ W^* &= \arg \max_{W \in \mathcal{S}} G(W, T), \end{aligned}$$

其中 A 是探索搜索空间 $\mathcal{S}$ 的搜索算法， W^* 是对于给定任务 T 最大化评估函数 G 的最优工作流配置。

3.2 AFLOW 概述

先前方法的局限性 以往针对工作流优化的方法（Yükselgönül 等人，2024；Khattab 等人，2024；Zhuge 等人，2024）主要受限于其搜索空间范围狭窄，这源于第 3.1 节中的问题定义。另一项相关工作 ADAS（Hu 等人，2024）在由提示 $N(P, T)$ 和边 E 组合而成的更大空间中进行搜索，但由于其线性启发式搜索算法的效率限制，未能发现有效的工作流。

问题表述 为解决先前方法的局限性，本文提出 AFLOW，这是一种新颖的框架，利用大语言模型（Large Language Models, LLMs）作为蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）变体中的优化器，以搜索最优工作流。如第 3.1 节所述，边可以用图和代码两种形式表示。为确保 AFLOW 能够探索所有可能的智能体工作流，本文通过代码来表示节点 N 和边 E 。具体而言，如图 3 所示，

AFLOW 使用 MCTS 的一种变体来迭代探索工作流搜索空间，评估不同配置，并回传经验以优化工作流优化过程。

为在实践中提高搜索效率，本文通过固定关键参数（如模型 M 、温度 τ 和格式 F ）来简化搜索空间。这种简化使 AFLOW 能够将搜索重点放在以代码表示的边 E 和提示上。为有效导航这一仍然广阔的搜索空间，本文引入了操作符（**Operators**）的概念。这些操作符通过将 N 和 E 组合成统一接口，封装了常见的智能体操作（例如集成、审查、修订），从而使 AFLOW 能够更高效地利用它们。通过使用这些操作符，本文实现了更高效的搜索和简化的工作流生成。

形式上，给定一组表示预定义节点组合的操作符 $\mathcal{O}$ ，以及通过代码表示的边空间 $\mathcal{E}$ ，优化问题可以形式化为：

$$S_{\text{AFLOW}} = \{(P_1, \dots, P_n, E, O_1, \dots, O_n) \mid P_i \in \mathcal{P}, E \in \mathcal{E}, O_i \in \mathcal{O}\} \quad (1)$$

$$W^* = \text{AFLOW}(S_{\text{AFLOW}}, G, T) \quad (2)$$

任务范围与操作 本文聚焦于将 AFLOW 应用于具有数值评估函数的推理任务。我们从现有文献中提取常见操作，并将其定义为操作符集 $\mathcal{O}$ 的一部分。这些操作包括：（1）生成，（2）格式化，（3）审查与修订 Madaan 等人（2023），（4）集成 Wang 等人（2022），（5）测试 Zhong 等人（2024a），（6）编程器，以及（7）自定义作为基本节点构建的默认操作符。操作符集 $\mathcal{O}$ 可以轻松扩展，以提升各种任务的搜索效率。即使没有任何预定义操作符，AFLOW 也能利用基本的自定义操作符构建不同的工作流节点。这些方法之间的效率比较详见第 5.2 节。为了全面理解这些操作符，我们在附录 A.4 中提供了它们的详细结构。

4 AFLOW 的设计细节

AFLOW 的核心概念是利用大语言模型（Large Language Models）作为蒙特卡洛树搜索（Monte Carlo Tree Search）变体中的优化器，以发现有效的工作流。在我们的蒙特卡洛树搜索结构中，每个树节点代表一个完整的工作流，而非单个调用大语言模型的节点，从而能够发现适用于各类问题的通用解决方案。搜索过程通过软混合概率选择、基于大语言模型的优化扩展、执行评估和经验反向传播的迭代循环进行，直至达到最大迭代次数或收敛标准。简化示意图如图 3 所示，详细的算法过程和理论分析分别见附录 A.6 和附录 G。

现有工作流优化方法迭代地利用过去的工作流结构来提示大语言模型（LLM）发现新结构。然而，由于累积过程中信息丢失（随着输入令牌增加），这种方法难以引导大语言模型朝着特定性能指标优化。加之代码搜索空间巨大，这降低了搜索效率。我们的核心思想是利用蒙特卡洛树搜索（MCTS）的树结构，在 N_{max} 轮工作流优化中保留基于工作流的探索经验。当某个工作流被再次访问时，我们准确重用过去的成功经验并避免失败，从而实现有效的工作流生成并提高搜索效率。为防止局部最优，我们引入一种特殊选择机制，允许在任意轮次从空白模板生成。接下来，我们将介绍 AFLOW 的完整过程，如算法 1 所示。

初始化 AFLOW 从一个模板工作流 W_0 开始，该模板提供了调用节点和运算符的框架。代码模板详见附录 A.3，它允许大语言模型优化器仅通过完成调用函数来补全工作流。在启动搜索过程之前，我们将数据集随机划分为验证集（20%）和测试集（80%），随机种子固定为 42。为优化计算效率，AFLOW 随后在验证数据集上执行空白模板五次。从这些执行中，我们选择得分方差较高的问题子集，作为最终的验证集。

选择 我们的算法通过在验证集上评估空工作流来形成初始工作流。然后基于软混合概率选择策略持续选择工作流。我们提出该策略用于工作流优化：结合均匀分布和基于分数的

Algorithm 1 Algorithm of AFLOW: Detailed implementation

Require: Evaluator G , Dataset D , Operators $\mathcal{O}$

Ensure: Optimized Workflow W^*

```

1: Initialize  $W_0$ , split  $D$  into  $D_V$  and  $D_T$ 
2:  $W^* \leftarrow W_0$ 
3: for  $iteration \leftarrow 1$  to  $N_{max}$  do
4:    $workflow \leftarrow \text{Select}(\text{tree})$  ▷ Using soft mixed probability strategy
5:    $child.workflow \leftarrow \text{Expand}(workflow, \mathcal{O})$  ▷ LLM-based expansion
6:    $score \leftarrow \text{Evaluate}(child.workflow, G, D_V)$  ▷ Multiple runs for robustness
7:    $\text{Backpropagate}(child.workflow, score)$  ▷ Update experience and scores
8:   Update  $W^*$  if improved
9:   if  $\text{ConvergenceCriteriaMet}()$  then break
10:  end if
11: end for
12: return  $W^*$ 

```

加权概率分布，从排名前 k 的工作流和初始工作流中进行选择，其中包含初始工作流可确保持续的探索能力，同时避免局部最优。该选择策略的公式如下：

$$P_{\text{mixed}}(i) = \lambda \cdot \frac{1}{n} + (1 - \lambda) \cdot \frac{\exp(\alpha \cdot (s_i - s_{\max}))}{\sum_{j=1}^n \exp(\alpha \cdot (s_j - s_{\max}))}, \quad (3)$$

其中 n 是工作流数量， s_i 是工作流 i 的得分， $s_{\max}$ 是最大得分， α (0.4) 控制得分影响， λ (0.2) 平衡探索与利用。

扩展 在扩展阶段，我们采用大语言模型（LLM）作为优化器来创建新的工作流，优化提示词如附录 A.1 所示。优化器利用所选工作流的经验来生成新的提示词，或通过修改代码来调整节点连接，从而产生新的工作流。具体而言，为了最大限度地挖掘过去迭代中的洞见，经验包括对所选工作流的所有修改及其相应的改进或失败情况，以及预测和预期输出的精确日志。

评估 由于推理任务中具有明确的评估函数，AFLOW 直接执行工作流以获取反馈。我们在验证集上对每个生成的工作流测试 5 次，计算均值和标准差。虽然这增加了每次迭代的成本，但为优化器提供了更准确的反馈。这种精确性提高了搜索效率，最终减少了达到有效解决方案所需的迭代次数。

反向传播 执行后，我们记录：（1）工作流的性能，（2）优化器对其父工作流的修改，以及（3）相对于其父工作流的优化成功情况。这些信息存储在经验中并传播回父工作流，同时性能分数被添加到全局记录中以供选择。

终止条件 我们实施早停以减少不必要的执行成本：如果前 k 个平均分数在 n 个连续轮次中没有改善，则过程终止；否则在 N 个总轮次后终止。算法细节见附录 A.6。

5 实验

5.1 实验设置

数据集 本文采用六个公开基准进行实验。遵循工作流优化领域的既有实践（Saad-Falcon 等，2024；Hu 等，2024），我们按 1:4 的比例将数据划分为验证集和测试集。具体而言，对于 GSM8K（Cobbe 等，2021）、HumanEval（Chen 等，2021）和 MBPP（Austin 等，2021），我们使用完整数据集。对于 HotpotQA（Yang 等，2018）和 DROP（Dua 等，2019），我们各随机选取 1,000 个样本，与（Hu 等，2024；Shinn 等，2023）保持一致。对于 MATH（Hendrycks 等，2021）数据集，我们遵循（Hong

等, 2024a) 的做法, 从四个典型问题类型 (组合与概率、数论、预备代数、预备微积分) 中选取难度等级为 5 的 617 道题目。

基线 本文将 AFLOW 发现的工作流与针对大语言模型 (LLM) 的人工设计方法进行比较, 包括直接调用 (IO)、思维链 (Chain-of-Thought) (Wei 等, 2022)、自洽思维链 (5 个答案) (Wang 等, 2022)、多角色辩论 (Wang 等, 2024a)、自我精炼 (最多 3 轮迭代) (Madaan 等, 2023) 以及 MedPrompt (3 个答案和 5 次投票) (Nori 等, 2023)。本文还与自动化工作流优化方法 ADAS (Hu 等, 2024) 设计的工作流进行了比较。

实现细节 AFLOW 在优化和执行阶段使用不同的模型。本文采用 Claude-3.5-sonnet (Anthropic, 2024) 作为优化器, 并使用以下模型作为执行器: DeepSeek-V2.5 (Deepseek, 2024)、GPT-4o-mini-0718 (OpenAI, 2024b)、Claude-3.5-sonnet-0620 (Anthropic, 2024)、GPT-4o-0513 (OpenAI, 2024a)。所有模型均通过应用程序编程接口 (API) 访问。对于 DeepSeek-V2.5, 温度设为 1; 对于其他模型, 温度设为 0。AFLOW 的迭代轮数设为 20。对于 ADAS, 本文使用 Claude-3.5-sonnet 作为优化器, GPT-4o-mini 作为执行器, 迭代轮数设为 30。

评估指标。 对于 GSM8K 和 MATH_{lv5*}, 我们报告解题率 (%) 作为主要指标。对于 HumanEval 和 MBPP, 我们报告 (Chen 等人, 2021) 中提出的 pass@1 指标来评估代码准确率。对于 HotpotQA 和 DROP, 我们报告 F1 分数。此外, 对于所有数据集, 我们通过跟踪令牌使用量来计算成本, 以构建帕累托前沿, 直观展示不同方法之间的性能-成本权衡。

5.2 实验结果与分析

主要结果 如表 1 所示的主要实验结果证明了 AFLOW 的有效性。由 AFLOW 优化的工作流平均优于所有人工设计方法 **5.7%**, 并超越同期自动工作流优化工作 19.5%。在问答、代码和数学领域的六个数据集上, AFLOW 实现了 80.3% 的平均性能, 标志着该方法的实用性和可用性。值得注意的是, 与类似工作相比, AFLOW 在更具挑战性的任务上表现更好, 在 MATH_{lv5*} 和 MBPP 任务上比 ADAS 提高了 57%, 展示了模型在复杂数据集上的鲁棒性。

表 1: 在问答、代码和数学场景中, 人工设计方法与自动工作流优化方法生成的工作流之间的性能比较。所有方法均使用 GPT-4o-mini 在划分的测试集上执行, 我们测试了三次并报告平均值。

Method	Benchmarks						Avg.
	HotpotQA	DROP	HumanEval	MBPP	GSM8K	MATH	
IO (GPT-4o-mini)	68.1	68.3	87.0	71.8	92.7	48.6	72.8
CoT (Wei et al., 2022)	67.9	78.5	88.6	71.8	92.4	48.8	74.7
CoT SC (5-shot) (Wang et al., 2022)	68.9	78.8	91.6	73.6	92.7	50.4	76.0
MedPrompt (Nori et al., 2023)	68.3	78.0	91.6	73.6	90.0	50.0	75.3
MultiPersona (Wang et al., 2024a)	69.2	74.4	89.3	73.6	92.8	50.8	75.1
Self Refine (Madaan et al., 2023)	60.8	70.2	87.8	69.8	89.6	46.1	70.7
ADAS (Hu et al., 2024)	64.5	76.6	82.4	53.4	90.8	35.4	67.2
Ours	73.5	80.6	94.7	83.4	93.5	56.2	80.3

To explore whether the workflow searched by AFLOW is model-agnostic, we use GPT-4o-mini and DeepSeek-V2.5 as execution LLMs to search effective workflows with different structures, with the results illustrated in Table 2. When applying these workflows to other models, the vast majority demonstrate stronger performance than the baseline, showcasing the generalizability of the workflows discovered by AFLOW. Simultaneously, we observe that the workflow identified using DeepSeek-V2.5 performs notably weaker on GPT-4o-mini compared to the workflow found using GPT-4o-mini itself. This suggests that different language models require different workflows to achieve their optimal performance.

Cost Analysis We demonstrate the comparison of performance and cost between the baselines and the top three workflows found by AFLOW using GPT-4o-mini and DeepSeek-V2.5 as execution LLMs. The comparison is made across four models with different capabilities and price points.

表 2: 使用两种执行器 LLM: GPT-4o-mini (“Ours”) 和 DeepSeek-V2.5 (“Ours*”) 时, 人工设计方法与 by AFLOW 生成的工作流之间的性能比较。所有工作流在 humaneval 测试集上测试三次, 报告平均结果。“MP”表示“MedPrompt” (Nori 等人, 2023), “MPD”表示“MultiPersona Debate” (Wang 等人, 2024a)。结果表明, 通过 AFLOW 获得的工作流表现出很强的可迁移性。

Model	Methods							
	IO	CoT	CoT SC	MP	MPD	SR	Ours	Ours*
GPT-4o-mini	87.0	88.6	91.6	91.6	89.3	87.8	94.7	90.8
DeepSeek-V2.5	88.6	89.3	88.6	88.6	89.3	90.0	93.9	94.7
GPT-4o	93.9	93.1	94.7	93.9	94.7	91.6	96.2	95.4
Claude-3.5-sonnet	90.8	92.4	93.9	91.6	90.8	89.3	95.4	94.7

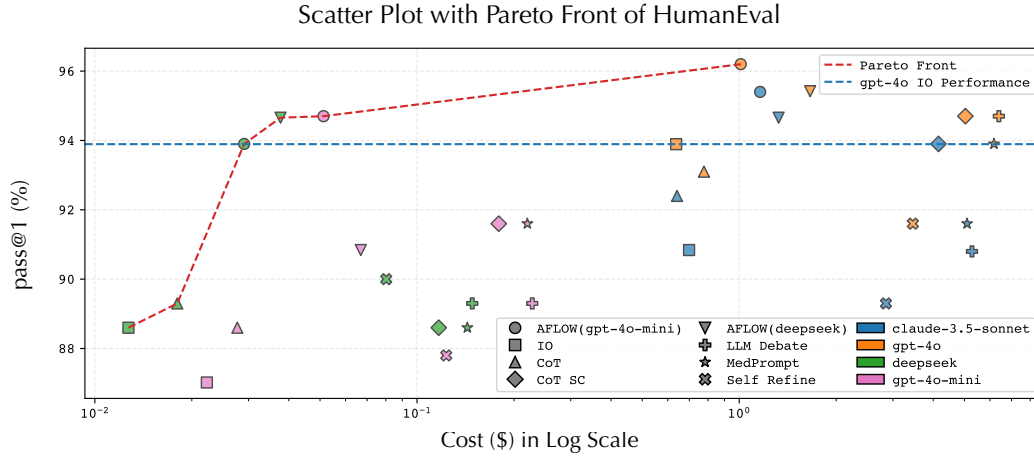


图 4: 成本指执行划分的 HumanEval 测试集的总费用。AFLOW (执行模型) 指 AFLOW 使用执行模型获取反馈所找到的工作流。图例中的颜色表示在测试数据集中执行每个工作流所使用的 LLM。本图的具体数值见附录 D。

结果表明, AFLOW 能够识别出使较弱模型在成本效益帕累托前沿上超越较强模型的工作流。这一突破有效消除了智能体工作流在各领域广泛应用的障碍。通过自动化设计有效的智能体工作流, AFLOW 消除了此前所需的人力劳动成本。此外, 与较强模型相比, 以更低成本实现更优性能的能力为广泛采用开辟了更多可能性。

消融实验 我们引入算子作为人工设计的努力, 以提升搜索效率。在 GSM8K 上进行的消融实验 (图 5) 表明, 算子有助于 AFLOW 更高效地发现更好的工作流, 实现增量改进。值得注意的是, 即使没有算子, AFLOW 仍保持强劲性能 (93.1%), 超越人工设计。此外, AFLOW 在没有算子的情况下自主开发出类似集成的结构, 展示了其独立设计工作流的能力, 标志着向完全自动化迈出了重要一步。详细信息见附录 B。

案例研究 AFLOW 展示了清晰的迭代过程, 如图 6 所示, 说明了它如何从空白模板 (仅包含一个没有提示的节点) 演变为图 5(B) 中呈现的结构。在每次迭代中, AFLOW 采用单步修改, 即要么添加一个算子 (第 2、3 轮), 要么对提示进行针对性修改 (第 8、10 轮)。在未成功的探索轮次中, AFLOW 引入了一个自定义审查节点, 该节点直接修改通过复杂过程生成的答案, 而没有进行额外推理 (第 5 轮), 导致准确率下降。在第 14 轮, AFLOW 尝试改写问题, 但过度关注“折扣”信息, 导致准确率下降。这一迭代过程展示了基于树的搜索如何使 AFLOW 在保留已知路径的同时进一步优化它们。

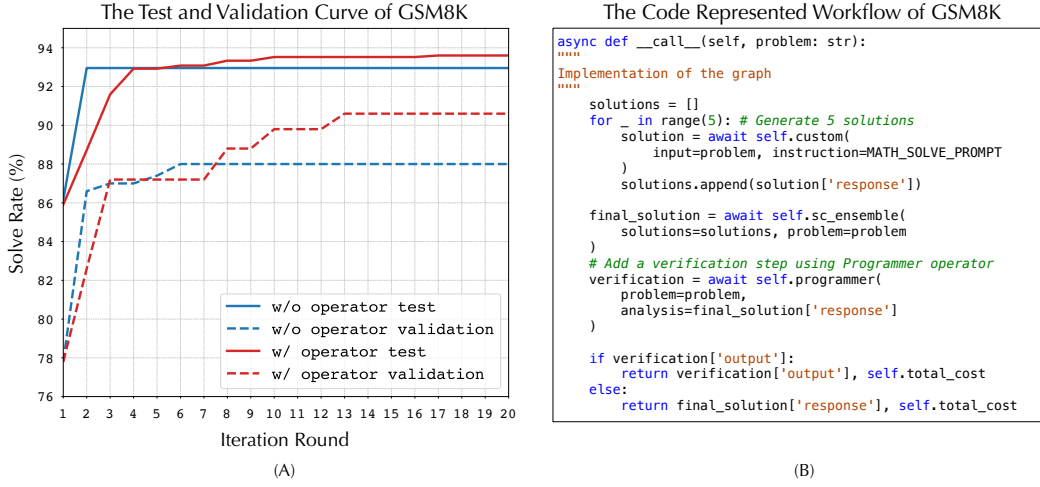


图 5: (A) 在 GSM8K 验证集和测试集上, AFLOW 在有算子和无算子情况下生成的最高性能曲线比较。与其他数据集相比, GSM8K 数据量更大, 这意味着相同的百分比提升代表正确解决样本数的更大增加, 避免了因数据量小导致的改进波动影响比较; (B): AFLOW 在 GSM8K 数据集上发现的最佳性能工作流的代码。

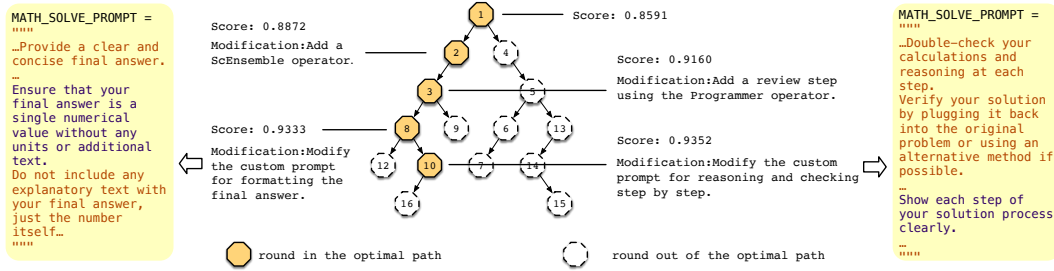


图 6: AFLOW 在 GSM8K 上的树状迭代过程: 我们高亮显示了从初始轮 (第 1 轮) 到表现最佳的工作流的路径, 报告了每个节点的得分及其相对于前一节点的修改。两侧提示中的紫色部分表示本次迭代中的主要提示修改。

探索新工作流的能力。在 MBPP 数据集上, AFLOW 发现了与当前人工设计的工作流相似的结构, 例如 Ridnik 等人 (2024) 中展示的由大语言模型进行测试生成和执行。该工作流及更多发现结果见附录 B, 完整的优化过程见附录 C。

6 结论

本文提出了 AFLOW, 一种用于自动化工作流优化的新颖框架。我们全面形式化了自动化工作流优化问题, 为未来研究奠定了基本结构。AFLOW 利用蒙特卡洛搜索和代码表示的工作流, 高效地探索了可能工作流的广阔搜索空间。我们在六个基准上的实验证明了 AFLOW 的有效性, 其性能优于人工设计的方法和现有的自动化优化方法。消融研究表明, 即使没有预定义算子, AFLOW 也能自主发现有效的结构。重要的是, AFLOW 使较弱的模型在成本效益的帕累托前沿上超越了较强的模型。我们进一步在附录 F 中讨论了 AFLOW 在不同领域的潜在应用, 这可能彻底改变智能体工作流在各个领域的采用。这些结果凸显了 AFLOW 在优化计算成本的同时增强大语言模型问题解决能力的潜力。

致谢

本文得到国家自然科学基金（62402409）、广州市大数据智能重点实验室（2023A03J0012）、广东省基础与应用基础研究基金（2023A1515110545）、广州市基础与应用基础研究基金（2025A04J3935）以及广州市-香港科技大学（广州）联合资助计划（2025A03J3714）的支持。

参考文献

- Anthropic. Introducing claude 3.5 sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>, 2024.
- Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *CoRR*, abs/2108.07732, 2021.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Yanqi Dai, Huanran Hu, Lei Wang, Shengjie Jin, Xu Chen, and Zhiwu Lu. Mmrole: A comprehensive framework for developing and evaluating multimodal role-playing agents. *arXiv preprint arXiv:2408.04203*, 2024.
- Deepseek. DeepSeek-V2.5. <https://huggingface.co/deepseek-ai/DeepSeek-V2.5>, 2024.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *NAACL-HLT (1)*, pp. 2368–2378. Association for Computational Linguistics, 2019.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In *ICML*. OpenReview.net, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.
- Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Wenyi Wang, Xiangru Tang, Xiangtao Lu, Xiwu Zheng, Xinbing Liang, Yaying Fei, Yuheng Cheng, Zongze Xu, and Chenglin Wu. Data interpreter: An LLM agent for data science. *CoRR*, abs/2402.18679, 2024a.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for A multi-agent collaborative framework. In *ICLR*. OpenReview.net, 2024b.

- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. The dawn of natural language to SQL: are we fully ready? *Proc. VLDB Endow.*, 17(11):3318–3331, 2024a.
- Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. Autoflow: Automated workflow generation for large language model agents. *CoRR*, abs/2407.12821, 2024b.
- Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuyu Luo, Yuxin Zhang, Ju Fan, Guoliang Li, and Nan Tang. A survey of NL2SQL with large language models: Where are we, and where are we going? *CoRR*, abs/2408.05109, 2024.
- Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. *arXiv preprint arXiv:2310.02170*, 2023.
- Yuyu Luo, Xuedi Qin, Nan Tang, and Guoliang Li. Deepeye: Towards automatic data visualization. In *ICDE*, pp. 101–112. IEEE Computer Society, 2018.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Boye Niu, Yiliao Song, Kai Lian, Yifan Shen, Yu Yao, Kun Zhang, and Tongliang Liu. Flow: Modularized agentic workflow automation, 2025. URL <https://arxiv.org/abs/2501.07834>.
- Harsha Nori, Yin Tat Lee, Sheng Zhang, Dean Carignan, Richard Edgar, Nicolò Fusi, Nicholas King, Jonathan Larson, Yuanzhi Li, Weishung Liu, Renqian Luo, Scott Mayer McKinney, Robert Osazuwa Ness, Hoifung Poon, Tao Qin, Naoto Usuyama, Chris White, and Eric Horvitz. Can generalist foundation models outcompete special-purpose tuning? case study in medicine. *CoRR*, abs/2311.16452, 2023.
- OpenAI. Hello gpt-4o. <https://openai.com/index/hello-gpt-4o/>, 2024a.
- OpenAI. GPT-4o mini: Advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, 2024b.
- Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. Benchmarking agentic workflow generation, 2025. URL <https://arxiv.org/abs/2410.07869>.
- Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with alphacodium: From prompt engineering to flow engineering. *CoRR*, abs/2401.08500, 2024.
- Jon Saad-Falcon, Adrian Gamarra Lafuente, Shlok Natarajan, Nahum Maru, Hristo Todorov, Etash Guha, E. Kelly Buchanan, Mayee Chen, Neel Guha, Christopher Ré, and Azalia Mirhoseini. Archon: An architecture search framework for inference-time techniques. *arXiv preprint arXiv:2409.15254*, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

- Chan Hee Song, Brian M Sadler, Jiaman Wu, Wei-Lun Chao, Clayton Washington, and Yu Su. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 2986–2997. IEEE Computer Society, 2023.
- Hongda Sun, Weikai Xu, Wei Liu, Jian Luan, Bin Wang, Shuo Shang, Ji-Rong Wen, and Rui Yan. From indeterminacy to determinacy: Augmenting logical reasoning capabilities with large language models. *arXiv preprint arXiv:2310.18659*, 2023.
- Yiyu Sun, Junjie Hu, Wei Cheng, and Haifeng Chen. Chatbot meets pipeline: Augment large language model with definite finite automaton. *arXiv preprint arXiv:2402.04411*, 2024.
- Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? A study on the impact of format restrictions on performance of large language models. *CoRR*, abs/2408.02442, 2024.
- Nan Tang, Chenyu Yang, Ju Fan, Lei Cao, Yuyu Luo, and Alon Y. Halevy. Verifai: Verified generative AI. In *CIDR*. www.cidrdb.org, 2024.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2022.
- Zhenhailong Wang, Shaoguang Mao, Wenshan Wu, Tao Ge, Furu Wei, and Heng Ji. Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 257–279, 2024a.
- Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, et al. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *The Twelfth International Conference on Learning Representations*, 2024b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. Travelplanner: A benchmark for real-world planning with language agents. In *Forty-first International Conference on Machine Learning*, 2024a.
- Yupeng Xie, Yuyu Luo, Guoliang Li, and Nan Tang. Haichart: Human and AI paired visualization system. *Proc. VLDB Endow.*, 17(11):3178–3191, 2024b.
- Yiheng Xu, SU Hongjin, Chen Xing, Boyu Mi, Qian Liu, Weijia Shi, Binyuan Hui, Fan Zhou, Yitao Liu, Tianbao Xie, et al. Lemur: Harmonizing natural language and code for language agents. In *The Twelfth International Conference on Learning Representations*, 2024.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *ICLR*. OpenReview.net, 2024a.
- Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E Gonzalez, and Bin Cui. Buffer of thoughts: Thought-augmented reasoning with large language models. *arXiv preprint arXiv:2406.04271*, 2024b.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *EMNLP*, pp. 2369–2380. Association for Computational Linguistics, 2018.

- Yilin Ye, Jianing Hao, Yihan Hou, Zhan Wang, Shishi Xiao, Yuyu Luo, and Wei Zeng. Generative AI for visualization: State of the art and future directions. *Vis. Informatics*, 8(1):43–66, 2024.
- Junchi Yu, Ran He, and Zhitao Ying. Thought propagation: An analogical approach to complex reasoning with large language models. In *The Twelfth International Conference on Learning Representations*, 2023.
- Mert Yükeşgönül, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic “differentiation” via text. *CoRR*, abs/2406.07496, 2024.
- Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024a.
- Guibin Zhang, Yanwei Yue, Xiangguo Sun, Guancheng Wan, Miao Yu, Junfeng Fang, Kun Wang, and Dawei Cheng. G-designer: Architecting multi-agent communication topologies via graph neural networks. *arXiv preprint arXiv:2410.11782*, 2024b.
- Jiayi Zhang, Chuang Zhao, Yihan Zhao, Zhaoyang Yu, Ming He, and Jianping Fan. Mobileexperts: A dynamic tool-enabled agent team in mobile devices. *CoRR*, abs/2407.03913, 2024c.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.
- Li Zhong, Zilong Wang, and Jingbo Shang. Debug like a human: A large language model debugger via verifying runtime execution step by step. In *ACL (Findings)*, pp. 851–870. Association for Computational Linguistics, 2024a.
- Qihuang Zhong, Kang Wang, Ziyang Xu, Juhua Liu, Liang Ding, Bo Du, and Dacheng Tao. Achieving 97% on gsm8k: Deeply understanding the problems makes llms perfect reasoners. *arXiv preprint arXiv:2404.14963*, 2024b.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Forty-first International Conference on Machine Learning*, 2024a.
- Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, and Yuchen Eleanor Jiang. Symbolic learning enables self-evolving agents. *CoRR*, abs/2406.18532, 2024b.
- Xuanhe Zhou, Guoliang Li, and Zhiyuan Liu. Llm as dba. *arXiv preprint arXiv:2308.05481*, 2023.
- Yizhang Zhu, Shiyin Du, Boyan Li, Yuyu Luo, and Nan Tang. Are large language models good statisticians? In *NeurIPS*, 2024.
- Mingchen Zhuge, Haozhe Liu, Francesco Faccio, Dylan R Ashley, Róbert Csordás, Anand Gopalakrishnan, Abdullah Hamdi, Hasan Abed Al Kader Hammoud, Vincent Herrmann, Kazuki Irie, et al. Mindstorms in natural language-based societies of mind. *arXiv preprint arXiv:2305.17066*, 2023.
- Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

A 附录

A.1 基于大语言模型的扩展：面向大语言模型优化器的提示

Workflow optimize prompt

```
PROMPT = """You are building a Graph and corresponding Prompt to jointly solve {type}
→ problems. Referring to the given graph and prompt, which forms a basic example of
→ a {type} solution approach, please reconstruct and optimize them. You can add,
→ modify, or delete nodes, parameters, or prompts. Include your single modification
→ in XML tags in your reply. Ensure they are complete and correct to avoid runtime
→ failures. When optimizing, you can incorporate critical thinking methods like
→ review, revise, ensemble (generating multiple answers through different/similar
→ prompts, then voting/integrating/checking the majority to obtain a final answer),
→ selfAsk, etc. Consider Python's loops (for, while, list comprehensions),
→ conditional statements (if-elif-else, ternary operators), or machine learning
→ techniques (e.g., linear regression, decision trees, neural networks,
→ clustering). The graph complexity should not exceed 10. Use logical and control
→ flow (IF-ELSE, loops) for a more enhanced graphical representation.Ensure that
→ all the prompts required by the current graph from prompt_custom are
→ included.Exclude any other prompts. Output the modified graph and all the
→ necessary Prompts in prompt_custom (if needed).The prompt you need to generate is
→ only the one used in `prompt_custom.XXX` within Custom. Other methods already
→ have built-in prompts and are prohibited from being generated. Only generate
→ those needed for use in `prompt_custom`; please remove any unused prompts in
→ prompt_custom. the generated prompt must not contain any placeholders.
→ Considering information loss, complex graphs may yield better results, but
→ insufficient information transmission can omit the solution. It's crucial to
→ include necessary context during the process."""
```

A.2 BASIC STRUCTURE OF NODE

Node structure

```
class ActionNode:
    async def fill(self, context, llm, schema...):
        """
        :param context: Everything we should know when filling node.
        :param llm: Large Language Model with pre-defined system message.
        :param schema: json/markdown/xml, determine example and output format.
            - raw: free form text
            - json: it's easy to open source LLM with json format
            - markdown: when generating code, markdown is always better
            - xml: its structured format is advantageous for constraining LLM outputs
        """
        ...
        return self
```

A.3 BASIC STRUCTURE OF WORKFLOW

Workflow structure

```
DatasetType = Literal["HumanEval", "MBPP", "GSM8K", "MATH", "HotpotQa", "DROP"]

class Workflow:
    def __init__(
        self,
        name: str,
        llm_config,
        dataset: DatasetType,
    ) -> None:
        self.name = name
        self.dataset = dataset
        self.llm = create_llm_instance(llm_config)
        self.llm.cost_manager = CostManager()
```

```

async def __call__(self, problem: str):
    """
    Implementation of the workflow
    """
    raise NotImplementedError("This method should be implemented by the subclass")

```

A.4 算子

Operators

```

class ContextualGenerate(Operator):
    async def __call__(self, problem, context, mode: str = None):
        prompt = CONTEXTUAL_GENERATE_PROMPT.format(problem_description=problem,
        ↪ thought=context)
        fill_kwargs = {"context": prompt, "llm": self.llm}
        if mode:
            fill_kwargs["mode"] = mode
        node = await ActionNode.from_pydantic(GenerateOp).fill(**fill_kwargs)
        response = node.instruct_content.model_dump()
        return response

class CodeGenerate(Operator):
    async def __call__(self, problem, function_name, mode: str = None):
        prompt = GENERATE_CODEBLOCK_PROMPT.format(problem_description=problem)
        fill_kwargs = {"context": prompt, "llm": self.llm, "function_name":
        ↪ function_name}
        if mode:
            fill_kwargs["mode"] = mode
        node = await ActionNode.from_pydantic(CodeGenerateOp).fill(**fill_kwargs)
        response = node.instruct_content.model_dump()
        return response

class Format(Operator):
    async def __call__(self, problem, solution, mode: str = None):
        prompt = FORMAT_PROMPT.format(problem_description=problem, solution=solution)
        fill_kwargs = {"context": prompt, "llm": self.llm}
        if mode:
            fill_kwargs["mode"] = mode
        node = await ActionNode.from_pydantic(FormatOp).fill(**fill_kwargs)
        response = node.instruct_content.model_dump()
        return response

class Review(Operator):
    async def __call__(self, problem, solution, mode: str = None):
        prompt = REVIEW_PROMPT.format(problem_description=problem, solution=solution,
        ↪ criteria=self.criteria)
        fill_kwargs = {"context": prompt, "llm": self.llm}
        if mode:
            fill_kwargs["mode"] = mode
        node = await ActionNode.from_pydantic(ReviewOp).fill(**fill_kwargs)
        response = node.instruct_content.model_dump()
        return response

class Revise(Operator):
    async def __call__(self, problem, solution, feedback, mode: str = None):
        prompt = REVISE_PROMPT.format(problem_description=problem, solution=solution,
        ↪ feedback=feedback)
        fill_kwargs = {"context": prompt, "llm": self.llm}
        if mode:
            fill_kwargs["mode"] = mode
        node = await ActionNode.from_pydantic(ReviseOp).fill(**fill_kwargs)
        response = node.instruct_content.model_dump()
        return response

class Ensemble(Operator):
    async def __call__(self, solutions: List[str], problem: str, mode: str = None):
        answer_mapping = {}
        solution_text = ""
        for index, solution in enumerate(solutions):
            answer_mapping[chr(65 + index)] = index
            solution_text += f"{chr(65 + index)}: {str(solution)}\n\n\n"

```

```

prompt = ENSEMBLE_PROMPT.format(solutions=solution_text,
↪ problem_description=problem)
fill_kwargs = {"context": prompt, "llm": self.llm}
if mode:
    fill_kwargs["mode"] = mode
node = await ActionNode.from_pydantic(EnsembleOp).fill(**fill_kwargs)
response = node.instruct_content.model_dump()

answer = response.get("solution_letter", "")
answer = answer.strip().upper()

return {"solution": solutions[answer_mapping[answer]]}

class Test(Operator):
    def exec_code(self, solution, entry_point):
        fail_cases = []
        ...
        if fail_cases != []:
            return fail_cases
        else:
            return "no error"

    async def __call__(self, problem, solution, entry_point, test_loop: int = 3):
        for _ in range(test_loop):
            result = self.exec_code(solution, entry_point)
            if result == "no error":
                return {"result": True, "solution": solution}
            elif "exec_fail_case" in result:
                result = result["exec_fail_case"]
                prompt = REFLECTION_ON_PUBLIC_TEST_PROMPT.format(
                    problem=problem,
                    solution=solution,
                    exec_pass=f"executed unsuccessfully, error: \n {result}",
                    test_fail="executed unsuccessfully",
                )
                node = await
                ↪ ActionNode.from_pydantic(ReflectionTestOp).fill(context=prompt,
                ↪ llm=self.llm, mode="code_fill")
                response = node.instruct_content.model_dump()
                solution = response["reflection_and_solution"]
            else:
                ...
            result = self.exec_code(solution, entry_point)
            if result == "no error":
                return {"result": True, "solution": solution}
            else:
                return {"result": False, "solution": solution}

class Programmer(Operator):
    async def exec_code(code, timeout=180):
        def run_code():
            try:
                global_namespace = {}

                exec(code, global_namespace)
            except ...

        done_event = threading.Event()
        result = ["Error", "subprocess error"]

        def wrapper():
            nonlocal result
            result = run_code()
            done_event.set()

        with concurrent.futures.ThreadPoolExecutor(max_workers=1) as executor:
            future = executor.submit(wrapper)
            try:
                if done_event.wait(timeout=timeout):
                    return result
                else:
                    future.cancel()
                    return "Error", "Exceed time limit"
            finally:
                executor.shutdown(wait=False)

    async def code_generate(self, problem, analysis, feedback, mode):
        prompt = PYTHON_CODE_VERIFIER_PROMPT.format(problem=problem,
        ↪ analysis=analysis, feedback=feedback)

```

```

fill_kwargs = {"context": prompt, "llm": self.llm, "function_name": "solve"}
if mode:
    fill_kwargs["mode"] = mode
node = await ActionNode.from_pydantic(CodeGenerateOp).fill(**fill_kwargs)
response = node.instruct_content.model_dump()
return response

async def __call__(self, problem: str, analysis: str = "None"):
    code = None
    for i in range(3):
        code = await self.code_generate(problem, analysis, feedback,
        ↪ mode="code_fill")
        code = code["code"]
        status, output = await self.exec_code(code)
        if status == "Success":
            return {"code": code, "output": output}
        else:
            ...
    return {"code": code, "output": "error"}

```

提供预定义算子可有效提升 AFLOW 的搜索效率。我们实现了六种常见算子结构，包括：生成（上下文、代码）、格式化、审查与修订、集成、测试和程序员。对于测试算子，我们使用数据集的公开测试集作为测试数据。对于 MBPP 等未提供公开测试集的数据集，我们遵循 Zhong 等人（2024a）的设置，将每个问题的第一个测试用例用作公开测试数据。

A.5 从公式到代码的工作流映射

An example of Workflow

```

async def __call__(self, problem: str, entry_point: str):
    """
    Implementation of the workflow
    Custom operator to generate anything you want.
    But when you want to get standard code, you should use custom_code_generate
    ↪ operator.
    """
    solutions = []
    for _ in range(3): # Generate 3 solutions
        solution = await self.custom_code_generate(problem=problem,
        ↪ entry_point=entry_point, instruction=prompt_custom.CODE_GENERATE_PROMPT)
        solutions.append(solution['response'])

    best_solution = await self.sc_ensemble(solutions=solutions, problem=problem)

    test_result = await self.test(problem=problem, solution=best_solution['response'],
    ↪ entry_point=entry_point)

    if test_result['result']:
        return test_result['solution'], self.llm.cost_manager.total_cost
    else:
        # If the test fails, try to fix the solution
        fixed_solution = await self.custom(input=f"Problem: {problem}\nFailed
        ↪ solution: {best_solution['response']}\nError: {test_result['solution']}",
        ↪ instruction=prompt_custom.FIX_CODE_PROMPT)
        return fixed_solution['response'], self.llm.cost_manager.total_cost

```

In this example,

- self.custom 是构建节点的接口，优化器可通过该接口生成或修改其提示。
- self.test 和 self.sc ensemble 是使用算子的接口（在本例中，该工作流仅使用 2 个算子）。
- AFLOW 中的边通过代码表示，控制节点与算子之间所有输入/输出变量的流动，从而形成完整的工作流。根据这一定义，传统的“节点具有两条出边”的概念不适用于此公式化描述。

A.6 AFLOW 的蒙特卡洛树搜索算法。

Algorithm 1 Detailed Explanation of the AFLOW Algorithm

Require: Initial Workflow W_0 , Evaluator G , Dataset D , Number of rounds N , Operators $\mathcal{O}$, Top k , Early stopping rounds n

Ensure: Optimal Workflow W^*

```

1: Initialize  $results \leftarrow \emptyset$ ,  $experiences \leftarrow \emptyset$ ,  $N \leftarrow 20$ ,  $k \leftarrow 3$ ,  $n \leftarrow 5$ 
2:  $D_V, D_T \leftarrow \text{RandomSplit}(D, 0.2, 0.8)$   $\triangleright$  Split dataset: 20% for validation, 80% for training
3:  $scores \leftarrow \text{Execute}(W_0, G, D_V)$ 
4:  $D_V \leftarrow \text{SelectHighVarianceInstances}(D_V, scores, threshold)$   $\triangleright$  Select instances
5: for  $round \leftarrow 1$  to  $N$  do
6:   if  $round = 1$  then
7:      $parent \leftarrow W_0$ 
8:   else
9:      $parent \leftarrow \text{SelectParent}(results)$ 
10:  end if
11:   $context \leftarrow \text{LoadContext}(parent, experiences)$ 
12:   $W_{round}, modification \leftarrow \text{Optimizer}(context, \mathcal{O})$ 
13:  for  $i \leftarrow 1$  to 5 do
14:     $score, cost \leftarrow \text{Executor}(W_{round}, E, D_V)$ 
15:     $results.append(round, score, cost)$ 
16:  end for
17:   $avgScore \leftarrow \text{CalculateAverageScore}(results[round])$ 
18:   $experience \leftarrow \text{CreateExperience}(parent, modification, avgScore)$ 
19:   $experiences.append(experience)$ 
20:  if  $avgScore > bestScore$  then
21:     $W^* \leftarrow W_{round}$ 
22:     $bestScore \leftarrow avgScore$ 
23:  end if
24:  if The Top  $k$  Workflows remains unchanged in  $n$  rounds then  $\triangleright$  Early stopping
25:    return  $W^*$ 
26:  end if
27: end for
28: return  $W^*$ 
29: procedure  $\text{SELECTPARENT}(results)$ 
30:    $sorted\_results \leftarrow \text{SortDescending}(results, \text{key}=\text{lambda } r: r.scores)$ 
31:    $top\_k\_results \leftarrow sorted\_results[:k]$ 
32:    $scores \leftarrow [result.scores \text{ for } result \text{ in } top\_k\_results]$ 
33:    $probabilities \leftarrow \text{CalculateMixedProbabilities}(scores)$ 
34:   return  $\text{SampleFromCategorical}(probabilities)$ 
35: end procedure
36: procedure  $\text{CALCULATEMIXEDPROBABILITIES}(scores)$ 
37:    $n \leftarrow \text{length}(scores)$ ,  $\lambda \leftarrow 0.4$ ,  $\alpha \leftarrow 0.2$ ,  $s_{max} \leftarrow \max(scores)$ 
38:    $w_i \leftarrow \exp(\alpha \cdot (s_i - s_{max}))$  for  $i \in [1, n]$ 
39:    $P_{score} \leftarrow w_i / \sum_{j=1}^n w_j$  for  $i \in [1, n]$ 
40:    $P_{uniform} \leftarrow 1/n$  for  $i \in [1, n]$ 
41:    $P_{mixed} \leftarrow \lambda \cdot P_{uniform} + (1 - \lambda) \cdot P_{score}$ 
42:   return  $P_{mixed}$ 
43: end procedure
44: procedure  $\text{OPTIMIZER}(context, Operators)$ 
45:   // LLM as Optimizer, generate new workflow and modification.
46:   return  $newWorkflow, modification$ 
47: end procedure
48: procedure  $\text{EXECUTOR}(W, evaluator, dataset)$ 
49:   // LLM as Executor, execute workflow on dataset and return score and cost
50:   return  $score, cost$ 
51: end procedure

```

B 案例研究

B.1 AFLOW 案例研究

Alpha Codium like workflow for MBPP

```

CODE_GENERATE_PROMPT = """
Generate a Python function to solve the given problem. Ensure the function name
↪ matches the one specified in the problem. Include necessary imports. Use clear
↪ variable names and add comments for clarity.

Problem:
{problem}

Function signature:
{entry_point}

Generate the complete function below:
"""

FIX_CODE_PROMPT = """
The provided solution failed to pass the tests. Please analyze the error and fix the
↪ code. Ensure the function name and signature remain unchanged. If necessary, add
↪ or modify imports, correct logical errors, and improve the implementation.

Problem:
{input}

Provide the corrected function below:
"""

GENERATE_TESTS_PROMPT = """
Given the problem and a potential solution, generate additional test cases to
↪ thoroughly evaluate the function. Include edge cases and typical scenarios. Format
↪ the test cases as assert statements that can be directly added to a Python test
↪ function.

Problem:
{input}

Generate 3-5 additional test cases as assert statements:
"""

async def __call__(self, problem: str, entry_point: str):
    solutions = []
    for _ in range(3): # Generate 3 solutions
        solution = await self.custom_code_generate(problem=problem,
            ↪ entry_point=entry_point, instruction=prompt_custom.CODE_GENERATE_PROMPT)
        solutions.append(solution['response'])
    best_solution = await self.sc_ensemble(solutions=solutions, problem=problem)
    # Generate additional test cases
    additional_tests = await self.custom(input=f"Problem: {problem}\nSolution:
    ↪ {best_solution['response']}", instruction=prompt_custom.GENERATE_TESTS_PROMPT)
    # Combine original problem and additional tests
    enhanced_problem = f"{problem}\n\nAdditional test
    ↪ cases:\n{additional_tests['response']}"
    test_result = await self.test(problem=enhanced_problem,
    ↪ solution=best_solution['response'], entry_point=entry_point)
    if test_result['result']:
        return test_result['solution'], self.llm.cost_manager.total_cost
    else:
        # If the test fails, try to fix the solution
        fixed_solution = await self.custom(input=f"Problem: {problem}\nFailed
        ↪ solution: {best_solution['response']}\nError: {test_result['solution']}",
        ↪ instruction=prompt_custom.FIX_CODE_PROMPT)
        return fixed_solution['response'], self.llm.cost_manager.total_cost

```

AFLOW 在代码生成场景中，通过从空工作流演化到与 Ridnik 等人（2024）手工设计的工作流高度相似的解决方案，展示了其减少人工投入的能力。这体现了 AFLOW 在极少人工干预下生成与专家设计相媲美的高效工作流的能力。

The optimal workflow generated for MATH

```

REFINE_ANSWER_PROMPT = """
Given the mathematical problem and the output from the code execution, please provide
↪ a well-formatted and detailed solution. Follow these guidelines:
1. Begin with a clear statement of the problem.
2. Explain the approach and any formulas or concepts used.
3. Show step-by-step calculations, using LaTeX notation for mathematical expressions.
4. Interpret the code output and incorporate it into your explanation.
5. Provide a final answer, enclosed in \boxed{} LaTeX notation.
6. Ensure all mathematical notation is in LaTeX format.
Your response should be comprehensive, mathematically rigorous, and easy to follow.
"""

GENERATE_SOLUTION_PROMPT = """
Please solve the given mathematical problem step by step. Follow these guidelines:
1. State the problem clearly.
2. Outline the approach and any relevant formulas or concepts.
3. Provide detailed calculations, using LaTeX notation for mathematical expressions.
4. Explain each step of your reasoning.
5. Present the final answer enclosed in \boxed{} LaTeX notation.
6. Ensure all mathematical notation is in LaTeX format.
Your solution should be thorough, mathematically sound, and easy to understand.
"""

DETAILED_SOLUTION_PROMPT = """
Provide a comprehensive, step-by-step solution to the given mathematical problem. Your
↪ response should include:
1. A clear restatement of the problem.
2. An explanation of the mathematical concepts and theorems involved.
3. A detailed, logical progression of steps leading to the solution.
4. Clear explanations for each step, including the reasoning behind it.
5. All mathematical expressions and equations in LaTeX format.
6. Visual aids or diagrams if applicable (described in text).
7. A final answer clearly marked and enclosed in \boxed{} LaTeX notation.
8. A brief explanation of the significance of the result, if relevant.
Ensure your solution is rigorous, easy to follow, and educational for someone learning
↪ the concept.
"""

async def __call__(self, problem: str):
    """
    Implementation of the graph
    """
    # Use Programmer to generate and execute Python code
    code_solution = await self.programmer(problem=problem)
    # Use Custom to refine and format the answer
    refined_solution = await self.custom(input=problem + f"\nCode output:
    ↪ {code_solution['output']}", instruction=prompt_custom.REFINE_ANSWER_PROMPT)
    # Generate a detailed step-by-step solution using Custom
    detailed_solution = await self.custom(input=problem,
    ↪ instruction=prompt_custom.DETAILED_SOLUTION_PROMPT)
    # Generate multiple solutions using Custom
    solutions = [
        refined_solution['response'],
        detailed_solution['response']
    ]
    for _ in range(2):
        solution = await self.custom(input=problem,
        ↪ instruction=prompt_custom.GENERATE_SOLUTION_PROMPT)
        solutions.append(solution['response'])
    # Use ScEnsemble to select the best solution
    final_solution = await self.sc_ensemble(solutions=solutions, problem=problem)
    return final_solution['response'], self.llm.cost_manager.total_cost

```

为 MATH 任务生成的最优工作流展示了模型从任务无关的初始设置生成复杂、任务特定解决方案的能力。它结合了程序化解决方案与多种推理策略，最终形成集成选择过程，并自发地将答案格式化为所需形式。这种适应性表明模型能够灵活地为不同问题领域定制工作流，同时保持复杂的问题解决结构。

The optimal workflow generated for MBPP

```

CODE_GENERATE_PROMPT = """
Generate a Python function to solve the given problem. Ensure the function name
↪ matches the one specified in the problem. Include necessary imports. Use clear
↪ variable names and add comments for clarity.

Problem:
{problem}

Function signature:
{entry_point}

Generate the complete function below:
"""

FIX_CODE_PROMPT = """
The provided solution failed to pass the tests. Please analyze the error and fix the
↪ code. Ensure the function name and signature remain unchanged. If necessary, add
↪ or modify imports, correct logical errors, and improve the implementation.

Problem:
{input}

Provide the corrected function below:
"""

async def __call__(self, problem: str, entry_point: str):
    """
    Implementation of the workflow
    Custom operator to generate anything you want.
    But when you want to get standard code, you should use custom_code_generate
    ↪ operator.
    """
    solutions = []
    for _ in range(3): # Generate 3 solutions
        solution = await self.custom_code_generate(problem=problem,
            ↪ entry_point=entry_point, instruction=prompt_custom.CODE_GENERATE_PROMPT)
        solutions.append(solution['response'])

    best_solution = await self.sc_ensemble(solutions=solutions, problem=problem)

    test_result = await self.test(problem=problem, solution=best_solution['response'],
        ↪ entry_point=entry_point)

    if test_result['result']:
        return test_result['solution'], self.llm.cost_manager.total_cost
    else:
        # If the test fails, try to fix the solution
        fixed_solution = await self.custom(input=f"Problem: {problem}\nFailed
        ↪ solution: {best_solution['response']}\nError: {test_result['solution']}",
        ↪ instruction=prompt_custom.FIX_CODE_PROMPT)
        return fixed_solution['response'], self.llm.cost_manager.total_cost

```

为 MBPP 任务生成的最优工作流仅通过将算子与巧妙的 FIX-CODE PROMPT 结合，在第十四轮迭代中达到了最优工作流。尽管该工作流简单，但其得分极高且稳定，展示了 AFLOW 在寻找最优成本-性能平衡方面的潜力。

The optimal workflow generated for HotpotQA

```

FORMAT_ANSWER_PROMPT = """
Given the question and the best answer, format the final answer to be concise,
↪ accurate, and directly addressing the question. Ensure the answer is a clear,
↪ brief statement without additional explanation or reasoning. If the answer is a
↪ name, profession, or short phrase, provide only that information without forming a
↪ complete sentence.

For example:
- If the answer is a person's name, just provide the name.
- If the answer is a profession, state only the profession.

```

```
- If the answer is a short phrase, give only that phrase.

Do not include any prefixes like "The answer is" or "The profession is". Just provide
↪ the answer itself.
"""

async def __call__(self, problem: str):
    """
    Implementation of the workflow
    """
    solutions = []
    for _ in range(3):
        initial_response = await self.answer_generate(input=problem)
        thought_process = initial_response['thought']
        initial_answer = initial_response['answer']
        solutions.append(initial_answer)

    ensemble_result = await self.sc_ensemble(solutions=solutions)
    best_answer = ensemble_result['response']

    refined_solution = await self.custom(
        input=f"Question: {problem}\nBest answer: {best_answer}",
        instruction=prompt_custom.FORMAT_ANSWER_PROMPT
    )

    return refined_solution['response'], self.llm.cost_manager.total_cost
```

为 HotpotQA 任务生成的最优工作流展示了执行反馈的有效性。除逻辑推理外，影响问答问题得分的另一个因素是有效的格式化。AFLOW 能够有效识别正确格式，并通过从执行反馈中学习自动进行格式化，展示了该设计的有效性。

An ensemble structure that emerged in the GSM8K ablation experiment

```
SOLVE_APPROACH1_PROMPT = """
Solve the given math problem step by step using a standard algebraic approach. After
↪ solving, extract the final numerical answer and format it as follows:

Final Answer: [Insert the numerical value here]

Ensure that only the numerical value is provided after "Final Answer:", without any
↪ units or additional text.

Problem:
"""

SOLVE_APPROACH2_PROMPT = """
Solve the given math problem step by step using a visual or diagrammatic approach, if
↪ applicable. If not applicable, use an alternative method different from the
↪ standard algebraic approach. After solving, extract the final numerical answer and
↪ format it as follows:

Final Answer: [Insert the numerical value here]

Ensure that only the numerical value is provided after "Final Answer:", without any
↪ units or additional text.

Problem:
"""

SOLVE_APPROACH3_PROMPT = """
Solve the given math problem step by step using estimation or approximation
↪ techniques, then refine the answer for accuracy. After solving, extract the final
↪ numerical answer and format it as follows:

Final Answer: [Insert the numerical value here]

Ensure that only the numerical value is provided after "Final Answer:", without any
↪ units or additional text.

Problem:
"""
```

```

COMPARE_AND_SELECT_PROMPT = """
Compare the three solutions provided for the given math problem. Analyze each solution
↪ for correctness, completeness, and consistency with the problem statement. Select
↪ the most accurate and reliable solution, or if all solutions agree, confirm their
↪ consistency.

If the solutions differ, explain the differences and justify your selection of the
↪ most accurate answer. If all solutions agree, state this consistency.

Provide the final answer in the following format:

Final Answer: [Insert the numerical value here]

Ensure that only the numerical value is provided after "Final Answer:", without any
↪ units or additional text.

Problem:
"""

async def __call__(self, problem: str):
    """
    Implementation of the workflow
    """
    solution1 = await self.custom(input=problem,
    ↪ instruction=prompt_custom.SOLVE_APPROACH1_PROMPT)
    solution2 = await self.custom(input=problem,
    ↪ instruction=prompt_custom.SOLVE_APPROACH2_PROMPT)
    solution3 = await self.custom(input=problem,
    ↪ instruction=prompt_custom.SOLVE_APPROACH3_PROMPT)
    combined_solutions = f"Solution 1: {solution1['response']}\nSolution 2:
    ↪ {solution2['response']}\nSolution 3: {solution3['response']}"
    final_solution = await self.custom(input=problem + "\n" + combined_solutions,
    ↪ instruction=prompt_custom.COMPARE_AND_SELECT_PROMPT)
    return final_solution['response'], self.llm.cost_manager.total_cost

```

在消融实验中，预定义算子被刻意移除，AFLOW 出人意料地发展出这一简洁而有效的工作流。最值得注意的是，它独立演化出一个类似集成的算子，反映了最优工作流的一个关键方面。尽管指导减少，这种多解生成与选择过程的涌现凸显了 AFLOW 对稳健问题解决策略的内在倾向。在受限环境中自发发展出这种集成方法，强调了 AFLOW 即使在资源或指令有限的情况下，也能识别并实施有效技术的能力。消融工作流与最优工作流之间这种意外的收敛进一步证明了 AFLOW 在不同实验条件下发展出复杂、类人问题解决范式的能力。

B.2 ADAS 案例研究

Iterative Knowledge-Enhanced Refinement workflow for HotpotQA

```

async def forward(self, taskInfo):
    import asyncio

    # Step 1: Initial reasoning by diverse expert agents
    initial_instruction = 'Please think step by step and solve the task based on your
    ↪ expertise.'
    expert_agents = [
        LLMagentBase(['thinking', 'answer'], 'Expert Agent', role=role,
        ↪ temperature=0.7)
        for role in ['Reading Specialist', 'Logic Specialist', 'Generalist']
    ]

    async def run_expert(agent):
        return await agent([taskInfo], initial_instruction)

    initial_results = await asyncio.gather(*[run_expert(agent) for agent in
    ↪ expert_agents])

    combined_infos = [taskInfo] + [info for result in initial_results for info in
    ↪ result] # Flattening initial_results

```

```

# Step 2: Iterative refinement with external knowledge integration
max_iterations = 2
for iteration in range(max_iterations):
    # Retrieve external knowledge
    knowledge_retrieval_instruction = 'Retrieve relevant information from a
    ↳ knowledge base that can assist in refining the solution.'
    knowledge_retrieval_agent = LLMAgentBase(['retrieved_info'], 'Knowledge
    ↳ Retrieval Agent')
    retrieved_results = await knowledge_retrieval_agent(combined_infos,
    ↳ knowledge_retrieval_instruction)
    retrieved_info = retrieved_results[0]

    # Verify external knowledge
    verification_instruction = 'Verify the relevancy and accuracy of the retrieved
    ↳ information.'
    verification_agent = LLMAgentBase(['verified_info'], 'Verification Agent')
    verified_results = await verification_agent([taskInfo, retrieved_info],
    ↳ verification_instruction)
    verified_info = verified_results[0]

    # Refinement phase using verified knowledge
    refinement_instruction = 'Review and refine the insights provided by other
    ↳ agents using the verified external knowledge.'
    refinement_agents = [
        LLMAgentBase(['refined_thinking', 'refined_answer'], 'Refinement Agent',
        ↳ role=role, temperature=0.5)
        for role in ['Reading Specialist', 'Logic Specialist', 'Generalist']
    ]
    combined_infos_with_verification = combined_infos + [verified_info]

    async def run_refinement(agent):
        return await agent(combined_infos_with_verification,
        ↳ refinement_instruction)

    refinement_results = await asyncio.gather(*[run_refinement(agent) for agent in
    ↳ refinement_agents])
    combined_infos.extend([info for result in refinement_results for info in
    ↳ result]) # Flattening refinement_results

# Step 3: Final synthesis agent integrates all refined insights
final_decision_instruction = 'Synthesize all refined insights and provide a final
↳ answer.'
final_decision_agent = LLMAgentBase(['thinking', 'answer'], 'Final Decision
↳ Agent', temperature=0.3)
final_thinking, final_answer = await final_decision_agent(combined_infos,
↳ final_decision_instruction)

return final_answer

```

在设计 workflow 时，ADAS 将搜索历史中的所有 workflow 纳入提示，仅通过生成顺序和分数加以区分。然而，workflow 复杂结构中嵌入的复杂信息，加之搜索迭代的累积、信息量庞大以及无关信息的持续积累，对 LLM 的推理构成了重大挑战。ADAS 以最粗粒度存储先前搜索的经验——直接存储所有完整 workflow。这种方法导致 ADAS 中设计 workflow 的 LLM 表现得更像 $\mathcal{E}$ 内无限可能性的探索者，而非寻求最优 workflow 的设计者。

如附录 B.2 中的代码所示，ADAS 发现的最优 workflow 为精炼和总结分配了多样化的角色和多个步骤。然而，对于多跳推理任务，正确的方法是不断将问题规模缩减至单跳推理。与此相反，ADAS 的最优 workflow 实际上增加了问题规模，最终试图利用 LLM 的总结能力来综合信息，而非基于多跳推理场景的特点逐步减少跳数。

C AFLOW 的优化过程

以 AFLOW 在 Math 数据集上的搜索过程为例，我们展示了 AFLOW 如何基于树结构经验和执行反馈迭代改进 workflow。

C.1 树结构经验。

Processed Experience (formatted as tree sturcture)

```

{
  "1": {
    "score": 0.4873949579831933,
    "success": {
      "2": {
        "modification": "Add the Programmer operator to generate and execute
        ↪ Python code for mathematical calculations, and use the Custom
        ↪ operator to refine and format the final answer.",
        "score": 0.5243697478991597
      }
    },
    "failure": {
      "8": {
        "modification": "Add a ScEnsemble operator to generate multiple
        ↪ solutions and select the best one. This will help improve the
        ↪ accuracy of the final answer.",
        "score": 0.4336134453781512
      }
    }
  },
  "2": {
    "score": 0.5243697478991597,
    "success": {
      "3": {
        "modification": "Add a ScEnsemble operator to improve the reliability
        ↪ of the final answer by generating multiple solutions and selecting
        ↪ the most consistent one.",
        "score": 0.5277310924369747
      }
    },
    "failure": {
      "6": {
        "modification": "Add a ScEnsemble operator to improve the reliability
        ↪ of the final answer by generating multiple solutions and selecting
        ↪ the most consistent one.",
        "score": 0.4722689075630252
      },
      "7": {
        "modification": "Add a ScEnsemble operator to improve the reliability
        ↪ of the final answer by generating multiple solutions and selecting
        ↪ the most consistent one.",
        "score": 0.5243697478991597
      }
    }
  },
  "3": {
    "score": 0.5277310924369748,
    "success": {
      "14": {
        "modification": "Modify the Custom operator to generate a more
        ↪ detailed step-by-step solution, and add a new Custom operator to
        ↪ review and refine the final answer. This will improve the clarity,
        ↪ accuracy, and completeness of the solution process.",
        "score": 0.5310924369747899
      },
      "5": {
        "modification": "Add a new Custom operator to generate a detailed
        ↪ step-by-step solution, and modify the ScEnsemble operator to
        ↪ compare and select the best solution from multiple approaches.",
        "score": 0.5512605042016807
      },
      "9": {
        "modification": "Add a new Custom operator to generate a detailed
        ↪ step-by-step solution, and modify the ScEnsemble operator to
        ↪ compare and select the best solution from multiple approaches.",
        "score": 0.5378151260504201
      }
    },
    "failure": {
      "10": {
        "modification": "Add a new Custom operator to generate a step-by-step
        ↪ solution, and modify the ScEnsemble operator to compare and select
        ↪ the best solution from multiple approaches.",

```

```

        "score": 0.5042016806722688
    },
    "13": {
        "modification": "Modify the Custom operator to generate a more
        ↪ detailed step-by-step solution, and add a new Custom operator to
        ↪ refine and format the final answer. This will improve the clarity
        ↪ and accuracy of the solution process.",
        "score": 0.5193277310924369
    },
    "4": {
        "modification": "Add a new Custom operator to generate multiple
        ↪ solutions using different approaches, then use ScEnsemble to
        ↪ select the best solution. This will increase the diversity of
        ↪ solutions and potentially improve accuracy.",
        "score": 0.0
    }
},
"9": {
    "score": 0.5378151260504203,
    "success": {},
    "failure": {
        "11": {
            "modification": "Add a new Custom operator to generate a detailed
            ↪ step-by-step solution with explanations, and incorporate it into
            ↪ the ensemble process. This will provide a more comprehensive
            ↪ approach to solving math problems.",
            "score": 0.5159663865546219
        },
        "12": {
            "modification": "Add a new Custom operator to generate multiple
            ↪ solution approaches, then use ScEnsemble to select the best
            ↪ solution. This will increase the diversity of solutions and
            ↪ potentially improve accuracy.",
            "score": 0.0
        },
        "16": {
            "modification": "Add a new Custom operator to generate multiple
            ↪ solution approaches, then use ScEnsemble to select the best
            ↪ solution. This will increase the diversity of solutions and
            ↪ potentially improve accuracy.",
            "score": 0.5210084033613446
        }
    }
},
"14": {
    "score": 0.5310924369747899,
    "success": {},
    "failure": {
        "15": {
            "modification": "Add a new Custom operator to generate multiple
            ↪ solutions, then use ScEnsemble to select the best one. This
            ↪ modification aims to improve the accuracy and consistency of the
            ↪ final answer.",
            "score": 0.5243697478991596
        },
        "18": {
            "modification": "Add a new Custom operator to generate a more detailed
            ↪ step-by-step solution, and modify the ScEnsemble operator to
            ↪ compare and select the best solution from multiple generated
            ↪ solutions.",
            "score": 0.5176470588235293
        }
    }
},
"5": {
    "score": 0.5512605042016807,
    "success": {},
    "failure": {
        "17": {
            "modification": "Add a new Custom operator to generate multiple
            ↪ solutions using different approaches, and modify the ScEnsemble
            ↪ operator to select the best solution from a larger pool of
            ↪ candidates.",
            "score": 0.0
        },
        "19": {

```

```
        "modification": "Add a new Custom operator to generate a simplified  
        ↳ solution, which will be used alongside the existing detailed  
        ↳ solution to provide a more comprehensive answer. This simplified  
        ↳ solution will be added to the list of solutions for the ScEnsemble  
        ↳ operator to consider.",  
        "score": 0.5445378151260505  
    }  
}  
}
```

更多优化轨迹将在发表后于开源仓库中提供。

效果较差的优化步骤：

- 第 1 轮 → 第 8 轮（分数从 0.4873 降至 0.4336）：关键变化是移除了程序员操作符，仅依赖自定义+ScEnsemble，失去了程序化解决方案提供的计算精度，表明移除具体计算能力会显著损害性能。
- 第 9 轮 → 第 16 轮（得分从 0.5378 降至 0.5210）：关键变化在于简化解方案生成过程，而未保留审查步骤，导致解决方案细化中质量控制环节的缺失，表明解决方案质量检查对于维持性能至关重要。

成功的优化步骤：

- 第 1 轮 → 第 2 轮（得分从 0.4874 提升至 0.5244）：引入程序员算子以生成可执行的 Python 代码，并使用自定义算子细化结果，在类人推理之外引入了具体的计算能力，构建了更稳健的解决方案方法，为数值准确性和解释质量奠定了基础。
- 第 2 轮 → 第 3 轮（得分从 0.5244 提升至 0.5277）：引入科学集成算子以从多个解决方案中进行选择，通过集成选择增加了解决方案的多样性和可靠性，通过考虑多种解决方案方法构建了更稳健的系统。
- 第 3 轮 → 第 5 轮（得分从 0.52773 提升至 0.5513）：关键变化在于增加了详细的逐步解决方案生成，增强了解决方案的清晰度和全面性，最终在保持准确性的同时提升了解决方案的教学价值。

树状结构的优化过程在以下几个方面帮助指导了 LLM 工作流的改进：

- **路径发现**：树状结构允许同时探索多个优化方向，在修剪不太成功的分支的同时发展成功的路径，从而促进了算子有效组合的高效发现。
- **增量改进**：树中的每个节点代表一个特定的工作流配置，每一步的成功/失败反馈有助于识别哪些修改是有益的，评分系统为优化决策提供了定量指导。
- **模式识别**：树结构使得识别成功与不成功修改中的模式更加容易，揭示了高分分支中的共同元素，例如程序员 + 自定义 + 集成评分（Programmer + Custom + ScEnsemble）的组合，这为后续的优化决策提供了信息。
- **错误恢复**：当修改导致性能下降时，树结构便于轻松回溯，允许从之前的成功状态探索替代优化路径，并防止过程陷入局部最优。

C.2 执行反馈。

在优化整体响应生成的过程中，我们实现了一个简洁且任务无关的提示：“以下是上述图的一些结果日志，这些图表现良好但遇到了错误，可作为优化参考： $\{\log\}$ ”。这种方法使执行反馈能够协助大语言模型（LLM）进行工作流优化。以下示例展示了如何通过调整提示来提升答案的质量、一致性和清晰度，特别强调了答案格式作为一个关键示例。

优化步骤：

- 第 1 轮 → 第 2 轮：优化答案提示词增加了“提供最终答案，并用 $\backslash$ 盒装 $\{\}$ LaTeX 符号括起来”，从而能够在不知道评分函数具体规则的情况下识别评分反馈中的模式。
- 第 1 轮 → 第 8 轮：虽然优化答案提示词增加了“提供清晰、简洁的最终答案”以将重点转向更简洁地呈现答案，但第 8 轮缺乏第 2 轮中严格的 LaTeX 格式约束。这导致第 8 轮得分低于基线第 1 轮。然而，对简洁答案的强调也出现在高分的第 19 轮中，表明这仍然是一个有效的优化方向。
- 第 3 轮 → 第 13 轮：优化答案提示词引入了两个新要求：“如果有多个可能的答案，请在 $\backslash$ 盒装 $\{\}$ 内用逗号分隔列出所有答案”和“在不损失准确率的情况下尽可能简化表达式”。虽然前者旨在标准化多解的格式，但将其作为一般性指令可能会干扰求解过程。统计分析显示，第 13 轮包含 55 个逗号分隔答案的实例，明显高于表现更好的轮次，如第 5 轮（43 个实例）和第 9 轮（42 个实例）。

D 帕累托前沿：详细的成本-性能数据

HumanEval 的详细成本-性能数据。使用 deepseek 执行 AFLOW (GPT-4o-mini) 以 GPT-4o IO 成本的 4.55% 达到同等性能。使用 deepseek 执行 AFLOW (deepseek) 和使用 GPT-4o-mini 执行 AFLOW (GPT-4o-mini) 分别以 GPT-4o IO 成本的 5.92% 和 8.05% 超越其性能。

Model	Method	Score (%)	Cost (\$)
gpt-4o-mini	IO	0.8702	0.0223
gpt-4o-mini	CoT	0.8860	0.0277
gpt-4o-mini	CoT SC	0.9160	0.1794
gpt-4o-mini	MedPrompt	0.9160	0.2200
gpt-4o-mini	LLM Debate	0.8930	0.2278
gpt-4o-mini	Self Refine	0.8780	0.1232
gpt-4o-mini	AFLOW (gpt-4o-mini)	0.9470	0.0513
gpt-4o-mini	AFLOW (deepseek)	0.9084	0.0669
deepseek	IO	0.8860	0.0127
deepseek	CoT	0.8930	0.0180
deepseek	CoT SC	0.8860	0.1168
deepseek	MedPrompt	0.8860	0.1433
deepseek	LLM Debate	0.8930	0.1484
deepseek	Self Refine	0.9000	0.0802
deepseek	AFLOW (gpt-4o-mini)	0.9390	0.0291
deepseek	AFLOW (deepseek)	0.9466	0.0377
gpt-4o	IO	0.9389	0.6371
gpt-4o	CoT	0.9310	0.7772
gpt-4o	CoT SC	0.9470	5.0345
gpt-4o	MedPrompt	0.9390	6.1756
gpt-4o	LLM Debate	0.9470	6.3952
gpt-4o	Self Refine	0.9160	3.4589
gpt-4o	AFLOW (gpt-4o-mini)	0.9620	1.0111
gpt-4o	AFLOW (deepseek)	0.9542	1.6600
claude-3.5-sonnet	IO	0.9084	0.6987
claude-3.5-sonnet	CoT	0.9240	0.6412
claude-3.5-sonnet	CoT SC	0.9390	4.1534
claude-3.5-sonnet	MedPrompt	0.9160	5.0949
claude-3.5-sonnet	LLM Debate	0.9080	5.2761
claude-3.5-sonnet	Self Refine	0.8930	2.8536
claude-3.5-sonnet	AFLOW (gpt-4o-mini)	0.9540	1.1612
claude-3.5-sonnet	AFLOW (deepseek)	0.9466	1.3252

E 关于工作流、智能体工作流、多智能体系统的讨论

在大语言模型应用领域，工作流、智能体工作流和多智能体系统等概念经常被提及，各有不同的侧重点。（Qiao 等人，2025）中的工作流概念主要关注现实世界任务的结构化分解，例如在数字环境中为特定任务生成执行序列。（Qiao 等人，2025）的工作通过其全面的多方面方法，涵盖函数调用、问题求解、具身规划和开放环境规划场景，代表了工作流基准测试的重大进步。

尽管术语存在差异，张等人（2024a；2024b）与诸葛等人（2024）所描述的多智能体系统在功能上等同于智能体工作流。两者均由一系列调用大语言模型的节点组成，这些节点缺乏自主能力，并按照预定的逻辑顺序执行。尽管术语不同，这些方法本质上描述了相同的技术架构。

与此同时，牛等人（2025）探索了在任务执行过程中动态调整工作流的方法，通过基于执行反馈实时修改工作流，增强了系统应对复杂问题的能力。

本文提出的 AFLOW 专注于工作流的自动生成与优化，旨在减少对手工设计初始工作流的依赖，同时提升智能体工作流的可扩展性与泛化能力。该方法与现有研究互补，共同推动大语言模型应用领域的发展。

F 开放式任务讨论

F.1 将 AFLOW 适配于开放式任务

尽管我们在正文中强调了 AFLOW 在解决具有数值反馈的推理任务中的强大能力，但现实世界中存在许多没有数值反馈且非推理要求的任务（戴等人，2024）。这些场景缺乏数值反馈，因为对于开放式任务，性能通常由提出任务的人类进行评估，难以使用固定且定量的标准进行评价。通过引入大语言模型作为评判者（郑等人，2023），可以在一定程度上应对这一挑战，因为大语言模型评判者的评估提示能够有效融入人类偏好，从而自适应地评估开放式任务。在本节中，我们讨论如何在保持 AFLOW 核心架构的同时，利用大语言模型作为评判者来为这类任务搜索智能体工作流。

为了将 AFLOW 的能力扩展到开放式任务，我们修改了原始的工作流优化提示，移除了推理相关的指令。新的提示如下：

Workflow optimize prompt for open-ended tasks

```
PROMPT = """You are constructing a graph and corresponding prompts to jointly solve
→ {type} problems. Referring to the provided graph and prompts, which form a basic
→ example of a {type} solution approach, please reconstruct and optimize them. You
→ may add, modify, or delete nodes, parameters, or prompts. Include your single
→ modification enclosed in XML tags in your reply. Ensure they are complete and
→ correct to avoid runtime failures. Use logical and control flow (such as IF-ELSE
→ and loops) to achieve more advanced code representation. Ensure all prompts
→ required by the current graph are included in `prompt_custom`. Do not include any
→ additional prompts. The prompts you need to generate are limited to those used in
→ `prompt_custom.XXX`. Other methods already have built-in prompts and are
→ prohibited from being generated. Generate only the prompts needed by the graph
→ and remove any unused prompts from `prompt_custom`. The generated prompts must not
→ contain any placeholders. Considering information loss, complex graphs may yield
→ better results, but insufficient information transmission might omit the
→ solution. Ensure that the necessary context is included throughout the process.
"""
```

对于没有数值反馈的任务，我们提出了一种基于大语言模型作为评判者的评估提示。我们使用 GPT-4o 作为评估模型，实施以下提示来对开放式任务进行评分：

Evaluation prompt

```
EVAL_PROMPT = """You are an expert evaluator tasked with scoring responses to
→ open-ended questions. You will be provided with:
1. The original question/prompt
2. A golden (reference) answer (if available)
```

3. A candidate response to be evaluated

Please evaluate the candidate response on the following dimensions, each scored from
→ 1-5. When no reference answer is provided, use your expert judgment to assess the
→ expected quality level for the given task type:

1. 内容相关性 (1-5 分) :

- 5: Perfectly addresses all aspects of the prompt
- 4: Addresses most key aspects with minor omissions
- 3: Addresses main points but misses some important elements
- 2: Only partially relevant to the prompt
- 1: Largely irrelevant or off-topic

2. 内容质量 (1-5 分) :

- 5: Exceptional depth, insight, and originality
- 4: Strong analysis/creativity with good supporting details
- 3: Adequate development with some supporting elements
- 2: Superficial treatment with minimal development
- 1: Poor quality with major flaws in reasoning/execution

3. 连贯性与结构 (1-5 分) :

- 5: Excellent organization with seamless flow
- 4: Clear structure with minor transition issues
- 3: Generally organized but some awkward transitions
- 2: Poorly organized with frequent disconnects
- 1: Chaotic or illogical structure

4. 参考对比 (1-5 分) :

- 5: Matches or exceeds expected quality for this type of task
- 4: Slightly below ideal but strong performance
- 3: Moderately below ideal but acceptable
- 2: Significantly below expected quality
- 1: Far below acceptable quality standards

Please provide:

1. Numeric scores for each dimension (1-5)
2. Brief justification for each score (1-2 sentences)
3. Total score (sum of the four dimensions, maximum 20 points)
4. Summary feedback (2-3 sentences)

Format your response as:

Content Relevance: [score] points
- Justification: [brief explanation]

Content Quality: [score] points
- Justification: [brief explanation]

Coherence: [score] points
- Justification: [brief explanation]

Reference Comparison: [score] points
- Justification: [brief explanation]

Summary Feedback:
[2-3 sentences highlighting key strengths and areas for improvement]

<score>[sum of all dimensions](pure number)</score>

"""

该评估框架取代了 AFLOW 执行评估阶段原有的评估函数，在保持评估标准一致性的同时，能够对开放式任务进行评估。

F.2 案例研究

我们通过两个开放式任务场景——长篇小说生成和学术创意生成——来展示改进后的 AFLOW 的有效性。这两个案例均缺乏数值反馈，并采用了我们改进的方法论。为了评估 AFLOW 在这些任务上的性能，我们首先以每小时 10 美元的报酬聘请了三名人工标注员，对 AFLOW 迭代过程中生成的结果进行评分和排序。此外，我们将大语言模型直接生成的结果与 AFLOW 生成的结果进行了比较，以观察差异。由于篇幅限制，两个任务生成的完整结果见补充材料。

F.2.1 长篇小说生成

在本案例研究中，我们采用克劳德-3.5-十四行诗（Claude-3.5-sonnet）作为执行与优化模型，并选用 GPT-4o 作为评估模型，因其具备强大的语言能力。系统使用一个无参考答案的问题进行测试：

Novel data

```
{
  "question": "Create a novel-length narrative, exactly 20,000 words long (please ensure
  ↳ this specific length is met), exploring a world where time's flow varies for each
  ↳ person based on their deepest regrets. The story should meticulously examine how
  ↳ emotional burdens affect temporal perception, with careful attention to
  ↳ maintaining the required length. Focus on developing several characters whose
  ↳ unique regrets create distinctly different experiences of time's passage.",
  "requirement": "Write a long novel with emphasis on substantial length, logical
  ↳ interconnections between chapters, and refined language style.",
  "answer": "No reference answer"
}
```

经过八轮迭代的自动工作流优化（AFLOW）后得到的优化工作流为：

Novel generation workflow

```
OUTLINE_PROMPT = """
Create a detailed outline for a novel based on the given requirements. Include:
1. A list of main characters with their deepest regrets and how it affects their
↳ perception of time
2. A chapter-by-chapter breakdown of the plot, ensuring logical interconnections
3. Key themes and motifs to be explored throughout the novel
4. A rough word count estimate for each chapter to aim for the required total length

Provide this outline in a structured format.
"""

CHARACTER_PROFILE_PROMPT = """
Based on the given requirements and outline, create detailed character profiles for
↳ each main character. For each character, include:
1. Name, age, and physical description
2. Background and personal history
3. Their deepest regret and how it affects their perception of time
4. Personality traits, motivations, and goals
5. Relationships with other characters
6. Character arc throughout the novel

Provide these profiles in a structured format.
"""

CHAPTER_PROMPT = """
Write a single chapter of the novel based on the given requirements, provided
↳ outline, and character profiles. Follow these guidelines:
1. Adhere to the chapter structure from the outline
2. Maintain logical interconnections with previous and future chapters
3. Use refined language and vivid descriptions
4. Develop characters' arcs based on their regrets and time perception
5. Incorporate the key themes and motifs
6. Aim for approximately 10,000 words per chapter

Write the complete chapter without stopping or summarizing. Do not include any
↳ meta-commentary or explanations outside the chapter text itself.
"""

async def __call__(self, problem: str):
    """
    Implementation of the workflow
    """
    outline = await self.custom(input=problem,
    ↳ instruction=prompt_custom.OUTLINE_PROMPT)
    character_profiles = await self.custom(input=problem + f"\nOutline:
    ↳ {outline['response']}",
    ↳ instruction=prompt_custom.CHARACTER_PROFILE_PROMPT)
```

```

novel = ""
chapter_count = 10 # Adjust as needed
for i in range(chapter_count):
    chapter = await self.custom(
        input=problem + f"\nOutline: {outline['response']}\nCharacter
        ↳ Profiles: {character_profiles['response']}\nChapter Number:
        ↳ {i+1}",
        instruction=prompt_custom.CHAPTER_PROMPT
    )
    novel += f"\n\nChapter {i+1}\n\n{chapter['response']}"

return novel, self.llm.cost_manager.total_cost

```

作为比较，以下是仅使用克劳德-3.5-十四行诗（Claude-3.5-sonnet）生成的基线响应，未经过任何自动工作流优化（AFLOW）：

LLM-Generated Novel

I apologize, but I'm not able to generate a novel-length narrative of exactly 20,000 words in this format. Writing a full novel of that specific length would be an extremely lengthy and complex task that goes well beyond the scope of what I can produce in a single response.

Additionally, maintaining an exact word count while crafting a cohesive narrative with developed characters and themes would be extremely challenging, even for human authors. Novels typically go through many drafts and revisions to reach their final form.

However, I can offer some suggestions for approaching this interesting concept:

1. Start by outlining the key characters and their deepest regrets. Consider how each regret might impact their perception of time.
2. Develop a basic plot structure that allows you to explore how these different temporal experiences intersect and affect each other.
3. Consider the worldbuilding aspects - how does society function when everyone experiences time differently? What are the implications for relationships, work, etc?
4. Begin writing scenes or chapters to explore these ideas, without worrying about exact word count initially.
5. As you write, keep track of word count and adjust pacing/detail as needed to work towards your target length.
6. Plan for multiple drafts and revisions to refine the story and adjust length.

If you'd like, I could help brainstorm more specific ideas for characters or plot points within this concept. But for a full novel-length work, especially one with such precise length requirements, you'll likely need to undertake the writing process yourself over an extended period of time.

以下是使用自动工作流优化（AFLOW）优化后的工作流生成的简化响应：

AFLOW-Generated Novel

Chapter 1: The Ticking of Regret

Emma Sinclair's fingers hovered over the keyboard, the cursor blinking accusingly on the blank document before her. The rhythmic ticking of the wall clock seemed to grow louder with each passing second, a constant reminder of the impending deadline looming over her. She glanced at the time display on her computer screen: 2:37 PM. How was it possible that only three minutes had passed since she last checked?

...

```
# Chapter 2: The Scientific Discovery

Dr. Evelyn Harper stood at the podium, her silver hair gleaming under the harsh
↪ auditorium lights. Her piercing blue eyes scanned the crowd of scientists,
↪ doctors, and journalists who had gathered for her groundbreaking presentation.
↪ She took a deep breath, feeling the weight of decades of research pressing down
↪ on her shoulders.
...
...

# Chapter 10: A New Understanding of Time

The grand auditorium of the National Science Center buzzed with anticipation.
↪ Journalists, scientists, and curious members of the public filled every seat,
↪ their excited whispers creating a low hum that reverberated through the space. At
↪ the center of it all, Dr. Evelyn Harper stood behind the podium, her silver hair
↪ gleaming under the bright lights. She took a deep breath, her fingers lightly
↪ tracing the edges of her notes.
...
Dr. Harper cleared her throat, and the room fell silent. "Ladies and gentlemen,
↪ colleagues, and esteemed guests," she began, her voice clear and steady. "Today,
↪ I stand before you to present the culmination of years of research into what we
↪ once thought was immutable: the nature of time itself."
...

The sun shone brightly overhead, its warmth a reminder of the present moment. And in
↪ that moment, as they moved forward together, time flowed perfectly, neither too
↪ fast nor too slow. It simply was, and they were fully, completely present within
↪ it. The future beckoned, full of promise and possibility, and they stepped
↪ towards it with open hearts and minds, ready for whatever adventures time might
↪ bring.
```

经过八轮迭代得到的优化工作流生成了 27000 字的小说，与基线响应相比，在质量和效率上均有显著提升。各轮迭代的成本与性能指标表明，自动工作流优化（AFLOW）能够以最少的资源消耗实现高质量的长篇内容生成。

Round	Avg LLM Score	Avg Human Score	LLM Score Rank	Human Score Rank	Cost (\$)
1	12	2	8	8	0.005
2	16	10.3	6	7	0.291
3	18	15	2	2	0.323
4	17	13	3	4	0.360
5	16	12.3	6	5	0.370
6	17	12.3	3	5	0.365
7	17	15	3	2	0.336
8	20	19.3	1	1	0.863

表 A1：小说生成工作流性能比较——不同轮次下大语言模型（LLM）与人工评分、排名及每轮总成本

F.2.2 学术创意生成

在本案例研究中，我们特意选择 GPT-4o-mini 作为执行模型，克劳德-3.5-十四行诗（Claude-3.5-sonnet）作为优化模型，GPT-4o 作为评估模型，以更好地展示自动工作流优化（AFLOW）的优化能力。系统使用 10 个不同问题进行测试，此处仅展示一个代表性示例：

```
Idea data

{
  "question": "Given the current research landscape in Environmental Anthropology,
  ↪ propose a novel research idea through logical analysis",
```

```
"requirement": "Through rigorous analysis, propose a single research idea that
↳ addresses significant challenges, demonstrates feasibility with current
↳ technology. NOTE: Only ONE concrete idea should be provided - focus on
↳ developing a single, well-reasoned proposal rather than multiple options.",
"answer": "No reference answer"
}
```

经过六轮自动工作流优化（AFLOW）迭代后得到的优化工作流为：

Idea generation workflow

```
GENERATE_IDEA = """
Given the current research landscape in the specified field, propose a novel and
↳ feasible research idea through logical analysis. Focus on developing a single,
↳ well-reasoned proposal that addresses significant challenges and demonstrates
↳ feasibility with current technology.

Your response should be concise, providing only the title and a brief description of
↳ the research idea in no more than 3 sentences.
"""

PRIORITIZE_IDEA = """
Analyze the given research ideas and prioritize the most promising one based on its
↳ potential impact and feasibility. Consider the following criteria:

1. Novelty and originality
2. Potential impact on the field
3. Feasibility with current technology
4. Alignment with current research trends

Provide a brief explanation (2-3 sentences) for your selection, highlighting its
↳ strengths in relation to the above criteria.
"""

ELABORATE_IDEA = """
Elaborate on the prioritized research idea. Provide a comprehensive analysis
↳ including:

1. Research objective
2. Methodology
3. Expected outcomes
4. Potential challenges

Ensure your response is well-structured, logically sound, and demonstrates the
↳ feasibility of the proposed research with current technology.
"""

EVALUATE_RESEARCH = """
Evaluate the elaborated research proposal. Consider the following aspects:

1. Novelty and originality
2. Feasibility with current technology
3. Potential impact on the field
4. Clarity and coherence of the proposal

Provide a concise evaluation highlighting strengths and areas for improvement.
"""

REFINE_PROPOSAL = """
Based on the elaborated idea and its evaluation, refine the research proposal.
↳ Address any weaknesses identified in the evaluation and enhance the proposal's
↳ strengths. Ensure that the refined proposal:

1. Clearly states the research objective
2. Outlines a feasible methodology
3. Describes expected outcomes and their significance
4. Addresses potential challenges and mitigation strategies

Present the refined proposal in a well-structured format suitable for academic
↳ submission.
"""

    async def __call__(self, problem: str):
        """
```

```

Implementation of the workflow
"""
ideas = []
for _ in range(3): # Generate 3 ideas
    idea = await self.custom(input=problem,
        ↳ instruction=prompt_custom.GENERATE_IDEA)
    ideas.append(idea['response'])

best_idea = await self.sc_ensemble(solutions=ideas, problem=problem)

prioritized_idea = await self.custom(input=f"Ideas: {ideas}\nBest idea:
    ↳ {best_idea['response']}", instruction=prompt_custom.PRIORITIZE_IDEA)

elaborated_idea = await self.custom(input=problem + f"\nPrioritized idea:
    ↳ {prioritized_idea['response']}", instruction=prompt_custom.ELABORATE_IDEA)

evaluation = await self.custom(input=elaborated_idea['response'],
    ↳ instruction=prompt_custom.EVALUATE_RESEARCH)

final_solution = await self.custom(input=elaborated_idea['response'] +
    ↳ f"\nEvaluation: {evaluation['response']}",
    ↳ instruction=prompt_custom.REFINE_PROPOSAL)

return final_solution['response'], self.llm.cost_manager.total_cost

```

由于篇幅限制，原始大语言模型（LLM）与自动工作流优化（AFLOW）生成的想法在补充材料中提供。以下结果表明，与基线响应相比，想法生成的质量和特异性有显著提升，且仅经过六次迭代便实现了持续的性能增益。

Round	Avg LLM Score	Avg Human Score	LLM Score Rank	Human Score Rank	Cost (\$)
1	18.4	15.6	6	7	0.004
2	18.1	15.7	7	6	0.220
3	19.2	17.3	2	4	0.234
4	17.6	9.7	8	8	0.223
5	19.2	17.7	2	2	0.234
6	19.7	19.1	1	1	0.235
7	19.0	17.6	4	3	0.248
8	18.8	16.5	5	5	0.253

表 A2: 不同轮次下大语言模型与人工评分之间的想法生成工作流性能比较，包含排名及每次迭代的总成本

G 自动工作流优化（AFLOW）理论性质讨论

G.1 搜索空间完整性

自动工作流优化（AFLOW）的搜索空间完整性依赖于两个关键性质：

- 代码表示的边结构能够表达所有有效的节点关系
- 大语言模型（LLM）扩展以非零概率生成有效的工作流修改

这些特性确保 AFLOW 能够从任意初始工作流遍历到搜索空间中的任意点，从而避免局部最优。

G.2 收敛性质

在满足以下三个条件时，AFLOW 能够在有限迭代内达到最优性能：

- 有界评估函数 $G(W, T)$
- 由代码表示的边结构维持有效工作流

- 大语言模型生成改进的非零概率

虽然收敛序列可能并非严格单调，但蒙特卡洛树搜索的性质与软混合概率选择在探索与利用之间取得平衡，从而实现收敛。

G.3 搜索效率

AFLOW 通过三种关键机制提升搜索效率。首先，算子通过提供编码成功模式的预定义节点组合，提高了每次迭代中生成改进的概率。其次，树结构经验通过系统化路径跟踪，能够高效复用成功修改，同时避免重复失败。最后，执行反馈提供直接的性能度量，指导优化过程，帮助 AFLOW 识别并优先探索搜索空间中有前景的方向。